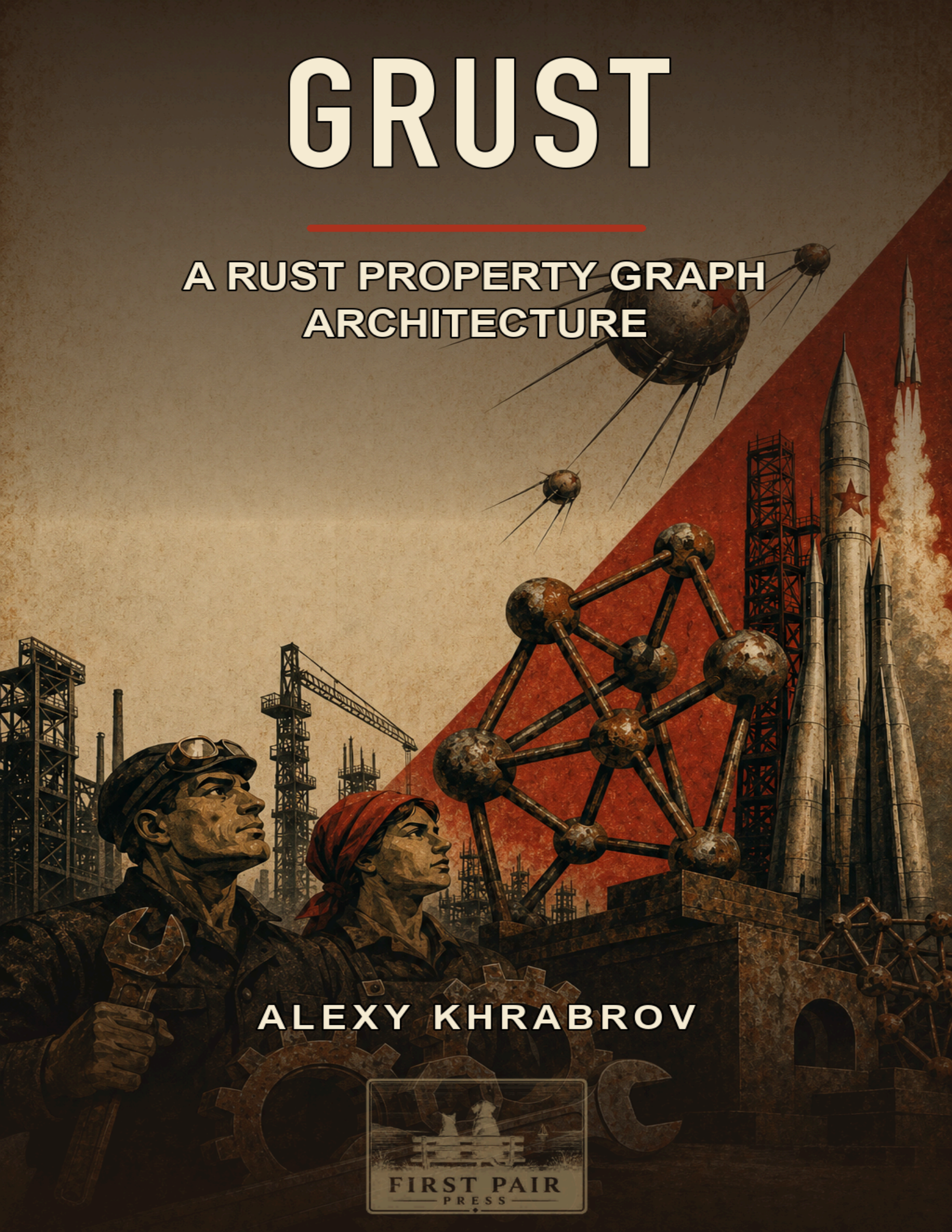
GRUST

A RUST PROPERTY GRAPH ARCHITECTURE



ALEXY KHRABROV

FIRST PAIR
PRESS

Grust

A Rust Property Graph Architecture

Alexy Khrabrov

2026-11-03

Version 0.24.1-f1d12c0a
Source commit f1d12c0af1ccb4bf4f32805db119d6902136d408

First Pair Press
firstpair.press

Contents

Preface 3
1 The Shape of Grust 4
2 The Core Property Graph 5
3 Building Graphs 6
3.1 Loading and Saving Graph Documents 7
4 Typed Graph Ingestion with garde and zod-rs 9
4.1 garde: Typed Rust Validation 9
4.2 Typed and Untyped Graphs Coexist 11
4.3 What Zod Means Here 13
4.4 How the Options Interact 15
4.5 Relationship to GraphSchema 17
5 Traversal as an Intermediate Representation 17
6 The Store Contract 18
7 Rust Concepts Used 19
8 Backend Architecture 19
8.1 Memory 20
8.2 LanceDB 21
8.3 LadybugDB 22
8.4 PostgreSQL and pgGraph 22
8.5 Turso 24
8.6 QueryGraph Memory: TypeSec over Grust 24
8.7 Sail 28
8.8 FalkorDB, HelixDB, and SurrealDB 30
8.9 CocoIndex 31
8.10 Backend Integration Tests 31
8.11 LSQB Compatibility Microbenchmark 32
9 Cypher and GQL 34
9.1 A portable read core 34
9.2 Values, procedures, transactions, writes 38
9.3 A backed profile 38
10 Example: A Conference Graph 39
11 Schema and Validation Direction 41
11.1 Semantic Model Projection 42
12 Design Tradeoffs 43
13 Where Grust Can Grow 44
14 Compact snapshots and multigraph persistence 45
15 Turso under strain 46
15.1 Two journal modes, two contracts 46
15.2 Loading an MVCC store 46
15.3 Concurrent writes and group commit 48
15.4 Allocator 49

15.5 What was measured 49
15.6 Methodology, because it decided several numbers 51
15.7 Limitations 51
16 Graph analytics through one Rust implementation 51
16.1 Starting from the facade 53
16.2 Available operations 54
16.3 Full paths and incremental consumption 60
16.4 Discovery, preparation reuse and execution classes 61
16.5 What the cooperative budget costs 61
16.6 Running without accounting 62
17 Graph algorithms under measurement 64
17.1 Execution classes, not engines 64
17.2 Full paths are an output contract 64
17.3 The cooperative budget was the kernel's largest cost 65
17.4 Deadline enforcement is a policy, and policies differ 65
17.5 Correctness is an oracle, not an afterthought 66
17.6 Allocator and durable loading 66
17.7 What the whole set moved 66
17.8 The same problem, one accounting axis over 67
17.9 What the unchanged columns say 68
17.10 Limitations 68
18 Arrow pipelines, ADBC and DataFusion 68
18.1 Native tables and readers 69
18.2 Property-graph interchange 69
18.3 Database ingestion 70
18.4 ADBC and extension 71
18.5 Typed Cypher execution 71
Conclusion 75

Preface

Grust is a small Rust property graph API with a large architectural promise: application code should be able to build, validate, traverse, and persist graph data without committing itself to a database query language too early.

That goal matters because many graph-shaped Rust applications live between worlds. A crawler wants a local graph while it extracts facts. An indexing pipeline wants deterministic target state. A data product may begin with tests and an in-memory backend, then later need SurrealDB, FalkorDB, PostgreSQL, LanceDB, Sail, or another backend. Grust gives those applications one model to program against:

```
Graph = nodes + edges
Node  = id + label + properties
Edge  = optional id + from + to + label + properties
```

This book is a guided tour of that model. It covers the workspace architecture, the Rust concepts the code leans on, the core graph types, traversal IR, backend contracts, implemented backend profiles, query safety, semantic-model projection, and practical examples.

1 The Shape of Grust

Grust is not trying to be every graph library for Rust. It is not an algorithm crate like `petgraph`, and it is not a thin wrapper around a single database. It is a property graph layer: a compact model of labeled nodes, labeled edges, and typed properties that can be carried across storage engines.

The current workspace is arranged around one core crate, one public facade, and backend or integration crates:

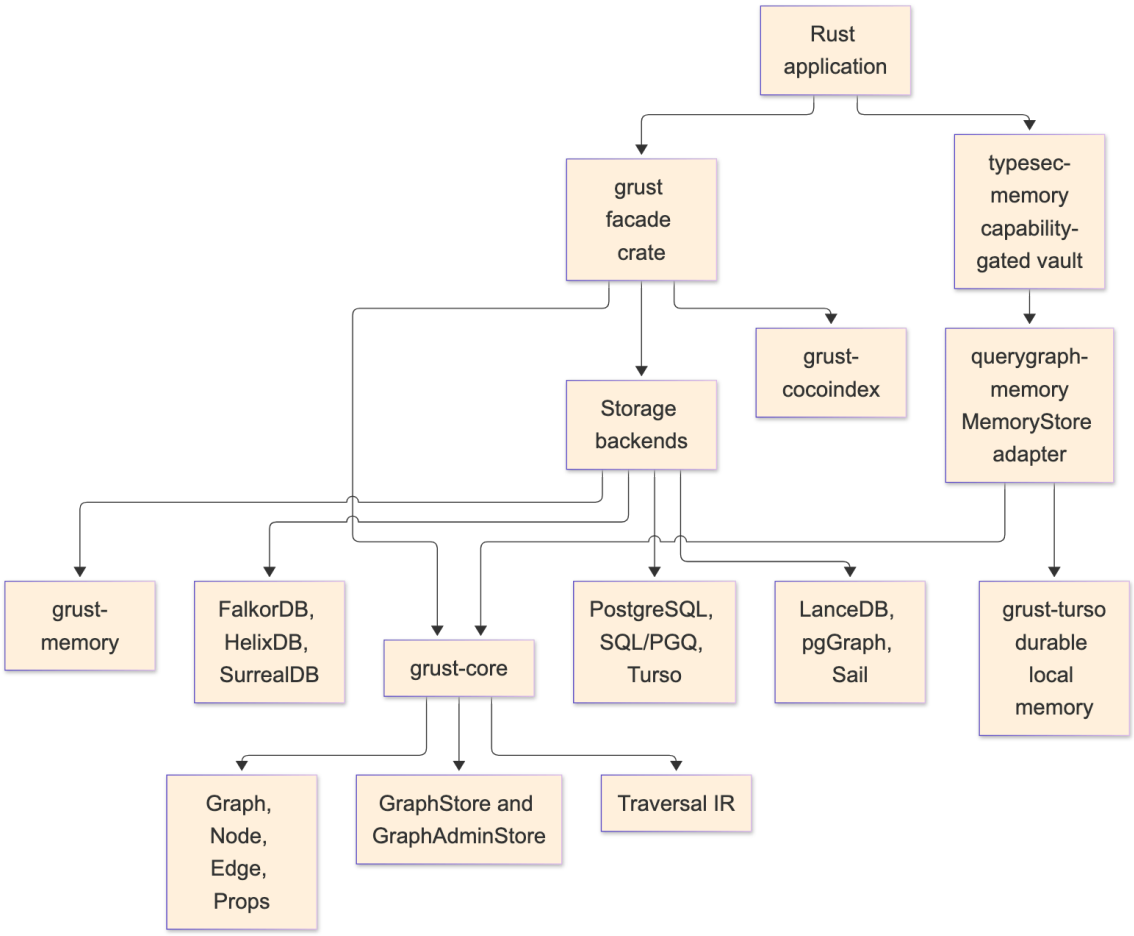


Figure 1: Diagram 1

The `grust-core` crate owns the durable concepts. It defines identifiers, labels, property values, graph builders, schemas, traversals, error types, load reports, and backend traits. The `grust` crate re-exports those pieces and gates backend crates behind Cargo features.

Core also owns the small lowering helpers that have to mean the same thing everywhere. `relationship_type` normalizes edge labels for backends that need database-safe relationship

names. `schema_identifier` normalizes schema labels and fields for SQL-like typed surfaces. `edge_key` gives table and export backends the same structural fallback key for an edge when the caller has not provided an explicit `EdgeId`.

That split is important. Core model code stays light, deterministic, and mostly dependency-free. Backends are allowed to depend on Redis, SurrealDB, LanceDB, PostgreSQL, Spark Connect, HTTP clients, gRPC clients, or SDKs without dragging those dependencies into every Grust user.

2 The Core Property Graph

At the center of Grust are three public data structures:

```
pub struct Graph {
    pub nodes: Vec<Node>,
    pub edges: Vec<Edge>,
}

pub struct Node {
    pub id: NodeId,
    pub label: Label,
    pub props: Props,
}

pub struct Edge {
    pub id: Option<EdgeId>,
    pub from: NodeId,
    pub to: NodeId,
    pub label: Label,
    pub props: Props,
}
```

`NodeId`, `EdgeId`, and `Label` are newtypes over `String`. They are small wrappers, but they carry meaning through Rust's type system. A function that expects a `Label` cannot accidentally receive an `EdgeId` without an explicit conversion. The wrappers implement common conversion traits such as `From<&str>` and `From<String>`, so the public API remains ergonomic.

Properties are stored in a `BTreeMap<String, Value>`:

```
pub enum Value {
    Null,
    Bool(bool),
    Int(i64),
    Float(f64),
    String(String),
    DateTime(RfcDate),
    Decimal(Decimal),
    Duration(Duration),
    StringArray(Vec<String>),
    IntArray(Vec<i64>),
    FloatArray(Vec<f64>),
```

```
    Path(PathValue),
    Graph(GraphValue),
    Json(serde_json::Value),
}
```

The use of `BTreeMap` is a quiet but useful choice. Iteration order is stable, which makes generated JSON, tests, and backend query generation more predictable. That is especially helpful in a project whose backends turn one graph into SQL, Cypher-like queries, Arrow batches, JSON target state, or SDK calls.

`GraphValue` deduplicates relationships by explicit ID or structural identity. Those identities use length-framed components rather than payload delimiters, so punctuation and control text inside an ID, label, or endpoint cannot make two distinct relationship values look identical. Persisted compatibility keys have a different contract: `checked_edge_key` rejects U+001F in every component before a tabular or export backend materializes the delimiter-based key.

`Node::new` also inserts an `id` property if one is not already present. That means a Grust node has a first-class typed identity and an ordinary property view of that identity, which helps backends that identify records through property maps.

3 Building Graphs

Most application code should not construct every `Node` and `Edge` by hand. It should use `GraphBuilder`:

```rust
use grust::prelude::*;

let mut builder = GraphBuilder::new();

let talk = builder
    .node("Talk", "talk:rust-graph-api")
    .prop("title", "A Modern Graph API for Rust")
    .prop("year", 2026_i64)
    .finish();

let speaker = builder
    .node("Person", "person:ada")
    .prop("name", "Ada Example")
    .finish();

builder
    .edge("PRESENTED_BY", &talk, &speaker)
    .prop("source", "conference-schedule")
    .finish();

let graph = builder.build();
```

The builder deduplicates nodes by `NodeId`. If an existing node has the same label, additional properties are merged into it. Edges default to `EdgePolicy::DedupeByFromLabelTo`, so the tuple

`(from, label, to)` is treated as the relationship identity. Domains that need true multi-edges can opt into `EdgePolicy::AllowDuplicates`.

```
let mut builder = GraphBuilder::new()
    .edge_policy(EdgePolicy::AllowDuplicates);
```

This is an example of Grust's overall style: put the common property-graph case on the simple path, but leave an explicit escape hatch for graph models with different identity rules.

3.1 Loading and Saving Graph Documents

A `Graph` can also move through textual document formats without touching a backend. The core crate exposes paired constructors and serializers for YAML, JSON, and XML:

```
let graph = Graph::from_yaml(yaml_text)?;
let yaml_text = graph.to_yaml()?;

let graph = Graph::from_json(json_text)?;
let json_text = graph.to_json()?;

let graph = Graph::from_xml(xml_text)?;
let xml_text = graph.to_xml()?;
```

These methods are useful for fixtures, migration inputs, examples, audits, and small graph interchange files. They all feed the same validation path, so a document with duplicate node ids or an edge that points at a missing node fails before it reaches a store.

YAML and JSON share the same document shape. A property value can be written as a plain JSON-like scalar when the type is obvious, or as the tagged Grust `Value` representation when the exact variant matters:

```
{
  "nodes": [
    {
      "id": "talk:rust-graph-api",
      "label": "Talk",
      "props": {
        "title": "A Modern Graph API for Rust",
        "year": 2026,
        "tracks": {
          "type": "string_array",
          "value": ["rust", "graphs"]
        }
      }
    },
    {
      "id": "person:ada",
      "label": "Person",
      "props": {
        "name": "Ada Example"
      }
```

```
    }
  ],
  "edges": [
    {
      "label": "PRESENTED_BY",
      "from": "talk:rust-graph-api",
      "to": "person:ada",
      "props": {
        "source": "conference-schedule"
      }
    }
  ]
}
```

The XML form is more explicit because XML has no native object or array type. Properties are represented as repeated `prop` entries with a `key` and a tagged `value`:

```
<graph>
  <nodes>
    <node>
      <id>talk:rust-graph-api</id>
      <label>Talk</label>
      <props>
        <prop>
          <key>title</key>
          <value>
            <type>string</type>
            <value>A Modern Graph API for Rust</value>
          </value>
        </prop>
      </props>
    </node>
    <node>
      <id>person:ada</id>
      <label>Person</label>
    </node>
  </nodes>
  <edges>
    <edge>
      <label>PRESENTED_BY</label>
      <from>talk:rust-graph-api</from>
      <to>person:ada</to>
    </edge>
  </edges>
</graph>
```

Serialization removes the generated `id` property from node property maps when it only mirrors the node's `NodeId`. That keeps exported documents readable without changing what `Node::new` guarantees after loading.

4 Typed Graph Ingestion with garde and zod-rs

The core `Graph`, `Node`, and `Edge` types are intentionally dynamic. A node has a label and a property map; an edge has a label, endpoints, and a property map. That dynamic shape is the right interchange format for backends, graph documents, audits, and cross-system movement. Application code, however, often starts with typed Rust structs: a `Person`, a `Project`, a `WorksOn` relationship, or some domain-specific event. Grust's typed ingestion layer connects those two worlds.

The typed layer is optional. It is enabled through Cargo features:

```
[dependencies]
grust = { package = "grust-graph", version = "0.24.1", features = ["typed-
garde"] }
```

`typed-garde` adds Rust-struct validation and typed lowering. A second feature, `typed-zod-rs`, layers raw JSON shape validation on top:

```
[dependencies]
grust = { package = "grust-graph", version = "0.24.1", features = ["typed-zod-
rs"] }
```

`typed-zod-rs` implies `typed-garde`. That relationship matters: zod-rs checks untrusted JSON shape first, Serde turns the JSON into Rust values, and garde checks typed domain invariants before Grust lowers the value into a normal `Node` or `Edge`.

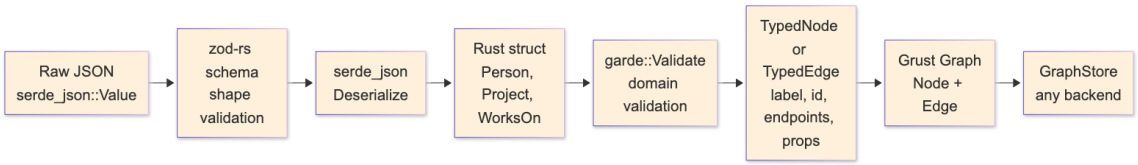


Figure 2: Diagram 2

The important promise is that typed ingestion does not create a second graph model. Once validation succeeds, everything becomes the ordinary Grust graph shape. Backends do not need to know whether a node came from `GraphBuilder`, a YAML document, a typed Rust struct, or a zod-rs-validated JSON payload.

4.1 garde: Typed Rust Validation

`garde` is a Rust validation library built around a derive macro. You attach validation rules to fields, derive `garde::Validate`, and then call `validate()` or `validate_with(...)`. The rules live near the Rust type, so they travel with the domain model instead of being hidden in a loader function.

Common garde rules include:

- `length(min = 1)` for nonempty strings and collections.
- `range(min = 1, max = 100)` for numeric bounds.
- `inner(...)` for validating items inside a collection.
- `custom(...)` for domain-specific validation functions.
- `dive` for validating nested values that also implement `Validate`.

In Grust, a typed node implements `TypedNode`. It supplies a graph label and a stable node id. The default property conversion serializes the struct through Serde and converts the resulting object fields into Grust properties:

```
use grust::prelude::*;
use grust::typed::garde;
use serde::Serialize;

#[derive(Debug, Serialize, garde::Validate)]
#[garde(allow_unvalidated)]
struct Person {
    #[garde(length(min = 1))]
    id: String,
    #[garde(length(min = 1))]
    name: String,
    #[garde(length(min = 1), inner(length(min = 1)))]
    skills: Vec<String>,
}

impl TypedNode for Person {
    const LABEL: &'static str = "Person";

    fn node_id(&self) -> NodeId {
        format!("person:{}", self.id).into()
    }
}
```

Typed edges implement `TypedEdge`. They provide a label and endpoint ids:

```
#[derive(Debug, Serialize, garde::Validate)]
#[garde(allow_unvalidated)]
struct WorksOn {
    #[garde(length(min = 1))]
    person_id: String,
    #[garde(length(min = 1))]
    project_id: String,
    #[garde(range(min = 1, max = 100))]
    allocation_percent: u8,
}

impl TypedEdge for WorksOn {
    const LABEL: &'static str = "WORKS_ON";

    fn source_node_id(&self) -> NodeId {
        format!("person:{}", self.person_id).into()
    }

    fn target_node_id(&self) -> NodeId {
        format!("project:{}", self.project_id).into()
```

```
    }
}
```

`TypedGraphBuilder` validates and lowers these values:

```
let mut builder = TypedGraphBuilder::new();

builder.add_node(&Person {
    id: "nia".to_string(),
    name: "Nia".to_string(),
    skills: vec!["rust".to_string(), "graphs".to_string()],
})?;

builder.add_edge(&WorksOn {
    person_id: "nia".to_string(),
    project_id: "grust".to_string(),
    allocation_percent: 80,
})?;

let graph = builder.build();
```

Validation fails before graph construction. If `allocation_percent` is `0`, the `range(min = 1, max = 100)` rule produces a Grust schema error and the edge is not added. This keeps invalid domain facts out of the graph rather than relying on a backend to reject them later.

Typed values can also be reconstructed from ordinary Grust reads. The read path decodes node or edge properties back through serde, runs garde validation, and checks that the typed identity still matches the graph value:

```
let stored = store
    .get_node(&NodeId::new("person:nia"))
    .await?
    .expect("person exists");

let person = Person::from_node(&stored)?;
assert_eq!(person.id, "nia");
```

Edges follow the same pattern with `WorksOn::from_edge(&edge)?`. For validation contexts, use `from_node_with` or `from_edge_with`.

4.2 Typed and Untyped Graphs Coexist

The typed layer is an ingestion layer over the ordinary `GraphBuilder`. It does not force an all-or-nothing choice. A graph can start as a document, accept raw nodes and edges, and then be extended with typed values:

```
let existing = Graph::new(
    vec![Node::new("Document", "doc:garde-proposal", Props::new())],
    Vec::new(),
);

let mut builder = TypedGraphBuilder::from_graph(existing);
```

```
builder.add_node(&Person {
    id: "nia".to_string(),
    name: "Nia".to_string(),
    skills: vec!["rust".to_string(), "graphs".to_string()],
})?;

builder.add_raw_edge(Edge::new(
    "AUTHORED",
    "person:nia",
    "doc:garde-proposal",
    Props::new(),
));

let graph = builder.build();
```

The coexistence API is explicit:

- `TypedGraphBuilder::from_graph(graph)` starts from an existing `Graph`.
- `TypedGraphBuilder::from_builder(builder)` starts from an existing `GraphBuilder`.
- `add_raw_node(node)` and `add_raw_edge(edge)` add ordinary Grust values.
- `into_builder()` returns the inner `GraphBuilder` when lower-level builder operations are needed.

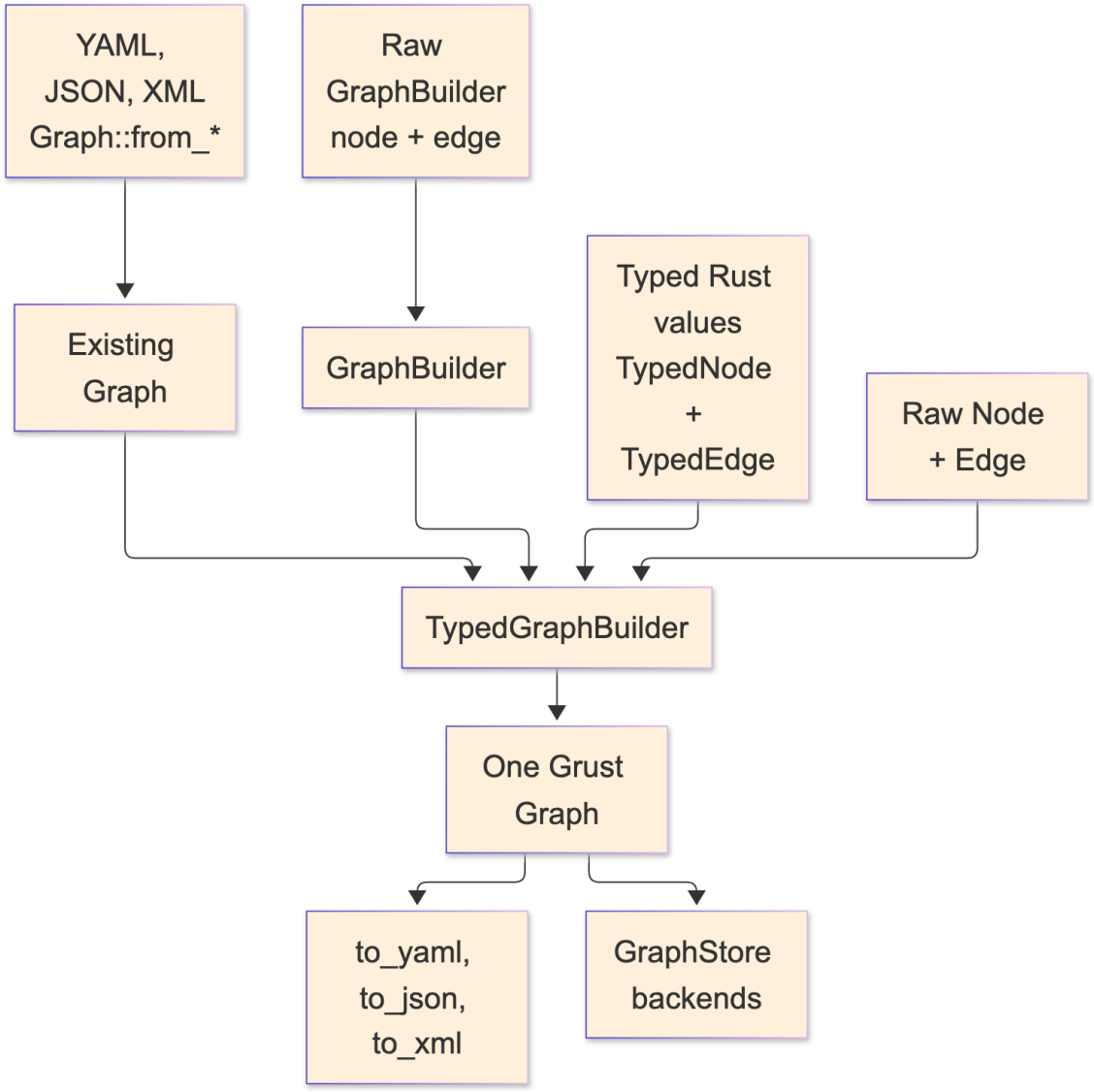


Figure 3: Diagram 3

This is useful during migrations. A project can keep loading existing graph documents while adding typed definitions for the domain facts that benefit most from Rust validation. Over time, more labels can move to typed constructors without breaking the storage or document formats.

4.3 What Zod Means Here

Zod is best known from TypeScript. It lets developers define a runtime schema and validate untrusted input before treating it as application data. In TypeScript, this closes an important gap: static types disappear at runtime, so a value received from JSON, an HTTP request, a form, or a message queue still needs runtime validation.

Rust has a different type system, but the boundary problem still exists. External data arrives as bytes or JSON. Serde can deserialize those bytes into a Rust struct, but it is often useful to separate two questions:

1. Does this JSON have the expected shape?
2. Does the resulting Rust value satisfy my domain rules?

`zod-rs` answers the first question. `garde` answers the second. Grust's `typed-zod-rs` feature wires those stages together:

```rust
use grust::prelude::*;
use serde::{Deserialize, Serialize};
use serde_json::json;
use zod_rs::prelude::{Schema, number, object, string};

#[derive(Debug, Deserialize, Serialize, garde::Validate)]
#[garde(allow_unvalidated)]
struct Person {
    #[garde(length(min = 1))]
    id: String,
    #[garde(length(min = 1))]
    name: String,
    #[garde(length(min = 1), inner(length(min = 1)))]
    skills: Vec<String>,
}

impl TypedNode for Person {
    const LABEL: &'static str = "Person";

    fn node_id(&self) -> NodeId {
        format!("person:{}", self.id).into()
    }
}

let person_schema = object()
    .field("id", string().min(1))
    .field("name", string().min(1))
    .field("skills", string().min(1).array())
    .strict();

let json = json!({
    "id": "nia",
    "name": "Nia",
    "skills": ["rust", "graphs"]
});

let person: Person = parse_typed_json(&person_schema, &json)?;
```

The same schema can feed the graph builder directly:

```rust
let mut builder = TypedGraphBuilder::new();
builder.add_node_from_json::<Person, _>(&person_schema, &json)?;
```

For edges, the pattern is the same:

```rust
#[derive(Debug, Deserialize, Serialize, garde::Validate)]
#[garde(allow_unvalidated)]
struct WorksOn {
    #[garde(length(min = 1))]
    person_id: String,
    #[garde(length(min = 1))]
    project_id: String,
    #[garde(range(min = 1, max = 100))]
    allocation_percent: u8,
}

impl TypedEdge for WorksOn {
    const LABEL: &'static str = "WORKS_ON";

    fn source_node_id(&self) -> NodeId {
        format!("person:{}", self.person_id).into()
    }

    fn target_node_id(&self) -> NodeId {
        format!("project:{}", self.project_id).into()
    }
}

let works_on_schema = object()
    .field("person_id", string().min(1))
    .field("project_id", string().min(1))
    .field("allocation_percent", number().int().min(1.0).max(100.0))
    .strict();

builder.add_edge_from_json::<WorksOn, _>(
    &works_on_schema,
    &json!({
        "person_id": "nia",
        "project_id": "grust",
        "allocation_percent": 80
    }),
)?;
```

The adapter intentionally deserializes the original JSON after zod-rs accepts it. This preserves Rust integer types. For example, `zod-rs` validates numbers through a floating-point schema, but Serde should still be allowed to decode the original JSON integer `80` into a Rust `u8`.

4.4 How the Options Interact

There are four common construction paths:

```
Raw GraphBuilder:
  trusted Rust code -> Node/Edge -> Graph

Graph documents:
  YAML/JSON/XML -> Graph::from_* validation -> Graph

typed-garde:
  Rust struct -> garde validation -> TypedNode/TypedEdge lowering -> Graph

typed-zod-rs:
  raw JSON -> zod-rs shape validation -> Serde -> garde validation -> Graph
```

Choose `GraphBuilder` when the data is already trusted Rust code and the dynamic graph model is enough. Choose graph documents when you need readable fixtures, interchange files, or migration inputs. Choose `typed-garde` when the domain model lives in Rust structs and should enforce Rust-level invariants. Choose `typed-zod-rs` when the input is untrusted JSON and you want a separate shape gate before Serde and garde.

These options compose. A single graph can contain nodes loaded from YAML, raw edges added by `GraphBuilder`, typed nodes validated by garde, and request payloads validated first by zod-rs. The result is still just `Graph`.

The distinction between zod-rs and garde is worth keeping crisp:

- zod-rs sees `serde_json::Value`.
- Serde converts JSON into Rust types.
- garde sees Rust fields with Rust types.
- `TypedNode` and `TypedEdge` convert validated Rust values into Grust labels, ids, endpoints, and properties.

For example, this JSON fails at the zod-rs stage because `skills` is not an array:

```
{
  "id": "nia",
  "name": "Nia",
  "skills": "rust"
}
```

This JSON can pass a loose zod-rs shape check but fail at the garde stage if the Rust type requires at least two skills:

```
{
  "id": "nia",
  "name": "Nia",
  "skills": ["rust"]
}
```

That separation gives application authors precise error boundaries. Shape errors usually belong near the API or file-ingest boundary. Domain errors belong near the typed model.

4.5 Relationship to GraphSchema

`GraphSchema` and typed ingestion solve different problems. `GraphSchema` describes graph labels, fields, uniqueness, and backend-facing metadata. It can drive indexes, migrations, table layouts, or validation inside a store. Typed ingestion validates values before they become graph data.

In practice they can reinforce each other:

- `TypedNode` and `TypedEdge` keep application construction honest.
- zod-rs schemas protect JSON ingress points.
- `GraphSchema` tells storage backends what structure to expect.
- `GraphStore` remains the common persistence contract.

This layered design keeps Grust from becoming either too loose or too rigid. Projects can start with raw graph construction, add typed Rust validation for important labels, add zod-rs for JSON ingress, and later add `GraphSchema` for backend optimization.

The same schema can also validate and write in one step:

```
let report = store.put_typed_graph(&schema, &graph).await?;
```

That call validates labels, required fields, field value types, and edge endpoint labels before delegating to the backend. Schema-capable stores then use `apply_schema` to lower the portable model into their native storage surfaces.

5 Traversal as an Intermediate Representation

Grust traversal is not Cypher, SQL, GQL, SurrealQL, Spark SQL, or a graph database dialect. It is a small Rust intermediate representation:

```
let traversal = Traversal::from_node("talk:rust-graph-api")
    .out("PRESENTED_BY")
    .to("Person")
    .limit(10);
```

A traversal has a start expression, a sequence of steps, and an optional limit. The start can be a single node, all nodes with a label, or nodes with a property value. Each step has a direction, an optional edge label, and an optional target node label.

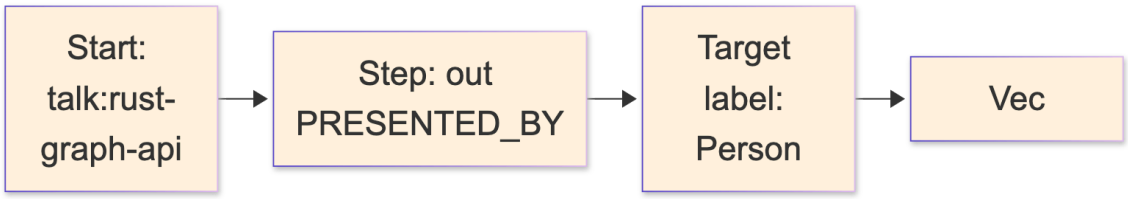


Figure 4: Diagram 4

The IR is deliberately modest. That makes it implementable across a wide range of backends. The memory backend can scan maps. LanceDB can query tables by hop. pgGraph can lower to SQL over universal graph tables. Sail can lower to Spark SQL through Spark Connect. Backends that do not yet support reads can return `GrustError::Unsupported`.

This design also protects application code. A caller asks for a graph-shaped operation; the backend decides whether that operation becomes a map scan, a SQL join, a DataFrame query, a Redis graph command, or a future native graph query.

For local analytics and backend planning, `grust-core` also exposes `GraphIndex`. It builds a dense vertex index from a `Graph`, validates that every edge endpoint exists, and stores incoming and outgoing edge indexes per vertex. That gives adapters and examples one shared adjacency layer instead of rebuilding node-id maps in each backend. The `grust-graph` facade includes a dependency-free `benchmarks` example that exercises graph cloning, `GraphIndex` construction, degree scans, endpoint scans, and structural edge-key generation over ring, grid, layered, clustered, Graph500-style R-MAT, and GAP-style R-MAT graph families.

6 The Store Contract

The central backend trait is `GraphStore` (capability and native-constraint methods are omitted here for brevity):

```
#[async_trait::async_trait]
pub trait GraphStore: Send + Sync {
    async fn apply_schema(&self, schema: &GraphSchema) -> Result<()>;
    async fn put_node(&self, node: &Node) -> Result<PutOutcome>;
    async fn put_edge(&self, edge: &Edge) -> Result<PutOutcome>;
    async fn put_graph(&self, graph: &Graph) -> Result<LoadReport>;
    async fn put_typed_graph(&self, schema: &GraphSchema, graph: &Graph) ->
Result<LoadReport>;
    async fn get_node(&self, id: &NodeId) -> Result<Option<Node>>;
    async fn get_nodes(&self, ids: &[NodeId]) -> Result<Vec<Node>>;
    async fn get_edges(&self, query: EdgeQuery) -> Result<Vec<Edge>>;
    async fn traverse(&self, traversal: Traversal) -> Result<Vec<Node>>;
}
```

The trait is async because real graph stores usually cross a process, network, database, or object-store boundary. The `async_trait` crate hides the current limitations around async functions in object-safe traits and lets backend implementations present one uniform interface.

`put_graph` takes `&Graph` instead of consuming `Graph`. That keeps retry, audit, comparison, and multi-backend loading workflows straightforward:

```
let report = store.put_graph(&graph).await?;
backup_store.put_graph(&graph).await?;
println!("loaded {} nodes and {} edges", report.nodes, report.edges);
```

Single-element writes return `PutOutcome`. Memory and builder paths can report precise inserted/updated/deduped outcomes. Remote upsert-oriented backends commonly return `Upserted` because distinguishing insert from update would require an extra read or a backend-specific write primitive. Portable callers should treat all written outcomes as success rather than depending on inserted-versus-updated. `LoadReport` counts elements that were written or upserted, not necessarily newly created rows.

Administrative operations are split into `GraphAdminStore`:

```rust
#[async_trait::async_trait]
pub trait GraphAdminStore: GraphStore {
    async fn bootstrap(&self) -> Result<()> { Ok(()) }
    async fn clear(&self) -> Result<()>;
}
```

This separation keeps ordinary application persistence apart from setup and destructive workflows. A production service may receive a `GraphStore`, while a test harness or migration tool can require `GraphAdminStore`.

7 Rust Concepts Used

Grust is a good example of idiomatic Rust applied to a storage abstraction.

Newtypes give semantic weight to strings. `NodeId`, `EdgeId`, and `Label` are cheap wrappers, but they prevent accidental parameter swaps and produce clearer APIs.

Trait-based polymorphism defines the backend boundary. Application code can be generic over `impl GraphStore`, while each backend owns its own connection, configuration, batching, serialization, and query strategy.

Feature flags keep the facade crate light. The public `grust` crate re-exports backend crates only when features such as `memory`, `lancedb`, `postgres`, `postgres-pgq`, `pggraph`, `turso`, `sail`, `falkor`, `surreal`, or `cocoindex` are enabled. The HelixDB and LadybugDB adapters remain internal `publish = false` workspace crates and are deliberately absent from the crates.io facade.

Serde makes the graph model portable. Core types derive `Serialize` and `Deserialize`, and backends can turn properties into JSON strings, JSONB, Arrow string columns, or target-state maps.

Interior mutability appears where it is appropriate. The memory backend stores its graph behind `Arc<RwLock<...>>`, making cloned stores share state while keeping mutation synchronized.

Error typing is explicit. `GrustError` distinguishes backend, schema, unsupported-feature, and serialization failures. That lets a caller tell the difference between "the database rejected this" and "this backend does not yet support traversal."

8 Backend Architecture

Backends share the same input model but not the same execution model:

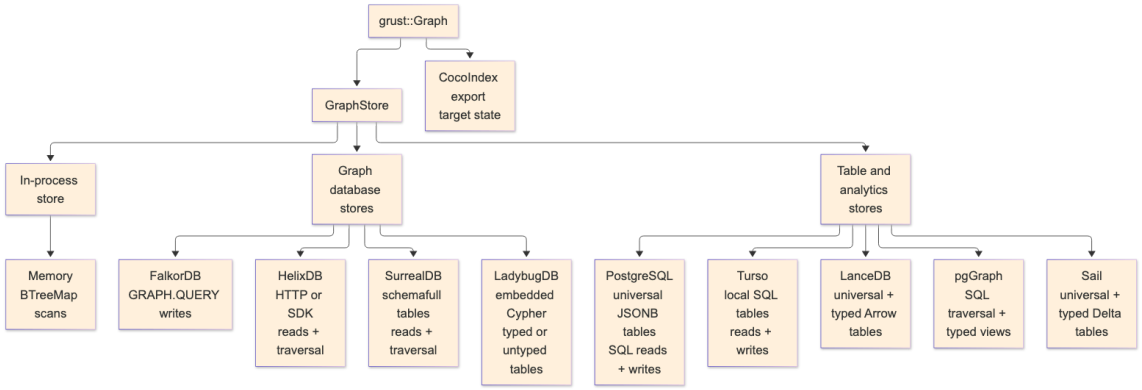


Figure 5: Diagram 5

Some backends are full read/write/traversal stores today. Others still focus on writes and administrative loading. That is normal for an early multi-backend project, and the trait makes the maturity boundary explicit.

8.1 Memory

`grust-memory` is the deterministic local backend. It interns node ids and labels as `u32` handles and stores each edge once, as a 16-byte record keyed by source, label, destination, and optional explicit edge ID, with ids and properties kept out of line. That means structural edges still behave deterministically, while id-bearing parallel edges between the same endpoints can coexist. Reads return nodes and edges in id and edge-key order. It is the best backend for tests, examples, and local workflows that need no external service. When a `GraphSchema` is applied, the memory backend validates writes against it, which makes it a useful conformance harness for typed storage behavior before a database enters the picture.

For repeated language reads, `indexed_snapshot()` returns a shared immutable `TypedGraphIndex`. The first call after a write builds typed incoming/outgoing adjacency over the store's own copy-on-write storage rather than a copy of the graph; later calls reuse it, including through cloned store handles. Every write attempt invalidates the cache, while snapshots already returned remain unchanged. Construction holds the store's read lock and therefore delays writers. Ordinary `GraphStore` reads and traversals keep their map-based behavior; indexed Cypher entrypoints are a separate choice.

```
use grust::prelude::*;

# async fn demo(graph: Graph) -> grust::Result<()> {
let store = MemoryGraphStore::new();
store.put_graph(&graph).await?;

let people = store
    .traverse(
        Traversal::from_node("talk:rust-graph-api")
            .out("PRESENTED_BY")
            .to("Person"),
```

```
    )
    .await?;
# Ok(())
# }
```

8.2 LanceDB

`grust-lancedb` treats LanceDB first as a durable Arrow-native table store. It uses two tables, one for nodes and one for edges. Nodes are keyed by `id`. Edges are keyed by an explicit edge id when present, or by a deterministic `from + label + to` key otherwise. That fallback key is shared with other tabular/export backends through `grust-core::edge_key`, so structural edge identity does not drift across implementations. Writes use LanceDB `merge_insert`, and traversal performs hop-by-hop table queries. When a hop fans out to multiple target nodes, the backend reads those target nodes with `get_nodes` instead of issuing one node query per edge. Property-start traversal filters by label in LanceDB, then compares decoded Grust properties exactly so nested JSON or serialized fragments cannot produce false positives. Before persisting either an explicit or structural edge key, LanceDB calls the checked identity path and rejects U+001F in the source ID, label, target ID, or explicit ID. CocoIndex, LadybugDB, Sail, and Cypher capture/refetch use the same guard, and mixed explicit/idless comparisons also require the same structural owner instead of trusting a key-shaped string alone. Ladybug's managed metadata index reserves the same delimiter, so its adapter rejects U+001F in node IDs before a user record can alias a table marker.

Schema object identity is checked separately. The shared `validate_physical_identifier_claims` helper lets FalkorDB, Helix, LadybugDB, LanceDB, and Sail reject lossy-name collisions and exact duplicate declarations within each native namespace before schema or write operations are emitted.

This backend is a natural home for later vector-search extensions. The core `GraphStore` trait should stay graph-focused; LanceDB-specific nearest-neighbor search can live in an extension trait without leaking into every backend.

With `GraphSchema`, LanceDB also creates typed Arrow tables per node and edge label. The universal `grust_nodes` and `grust_edges` tables remain the portable read/traversal surface, while schema-labeled rows are mirrored into tables such as `grust_node_person` or `grust_edge_presents` with typed columns for declared fields. That gives analytical consumers and future vector extensions a native columnar surface without giving up the backend-neutral graph model.

Mantis 0.19.0′s write path groups concurrent single-row writes through clones of one store into universal-table commits, preserving last-key-wins order inside a batch. Queue leadership is owned across cancellation and handoff. A cancelled leader can leave an in-flight batch's durability uncertain, which its waiting callers receive as an explicit error. Universal and typed-mirror writes remain separate commits; this is not an atomic multi-table transaction.

Bulk graph and shared Arrow loads compact their fragments and create or update merge-key indexes. Single-row commits periodically compact fragments, and writes release the store's retained read snapshot. A connection's index and metadata cache budgets are 512 MiB and 128

MiB. These are cache limits, not a total process memory bound. Performance evidence must disclose dataset, concurrency, indexing, load cost and peak-memory boundaries separately from correctness qualification.

8.3 LadybugDB

`grust-ladybug` embeds LadybugDB directly through the Rust `lbug` 0.20.4 crate. It is the durable local graph-database backend: no Docker service, no HTTP bridge, and no separate daemon. The store opens either an in-memory Ladybug database or an on-disk Ladybug directory and creates Grust-managed Ladybug node and relationship tables from graph labels.

Ladybug can be used for typed or untyped property graphs, and the Grust backend now exposes that distinction directly. The default `LadybugGraphMode::Untyped` accepts ordinary Grust graphs and creates the needed Ladybug node and relationship tables from graph labels and endpoint labels on write. That mode matches Grust's raw `Graph`, JSON, YAML, and XML loading path, where labels and properties are data owned by the application.

`LadybugGraphMode::Typed` requires `apply_schema` or `put_typed_graph` before writes. In that mode the backend creates the declared Ladybug tables up front, does not create undeclared tables during writes, and validates later writes against the applied `GraphSchema`. Reads reconstruct Grust `Node` and `Edge` values from the managed tables, while traversal evaluates the portable Grust traversal IR by walking Ladybug relationship tables and reading target nodes through the same `GraphStore` contract.

Ladybug's managed metadata index uses U+001F to frame table markers and node entries. The adapter therefore rejects that delimiter in node IDs before any mutation; this is stricter than the backend-neutral `NodeId` type and is part of the Ladybug storage contract.

The internal crate's `arrow` feature also exposes Ladybug's embedded Arrow table path through Arrow IPC streams. A workspace caller can register IPC node tables, relationship tables, and CSR relationship tables directly with Ladybug, then query them with Ladybug Cypher and receive result chunks back as Arrow IPC. The public boundary is IPC bytes rather than a Rust `RecordBatch` type, so callers do not have to match Ladybug's internal Arrow crate version exactly.

The first implementation stores Grust properties as JSON text for portable round trips. Later schema lowering can add typed Ladybug columns, full-text indexes, vector indexes, and direct graph-RAG extension traits without changing the core graph model.

8.4 PostgreSQL and pgGraph

`grust-postgres` stores the source of truth in ordinary PostgreSQL tables:

```
grust_nodes(id text primary key, label text not null, props jsonb not null)
grust_edges(id text, from_id text not null, to_id text not null,
            label text not null, props jsonb not null)
```

The generic backend creates tables and indexes with ordinary PostgreSQL DDL. It does not require PostgreSQL extensions, so the same `PostgresGraphStore` can target local PostgreSQL, Neon, or another managed PostgreSQL-compatible service. Reads use SQL against the universal tables.

Traversal is lowered to SQL joins over those tables. Mutation batches are wrapped in PostgreSQL transactions.

The shared connection is serialized across every explicit transaction. A recovery marker is set before `BEGIN` and cleared only after PostgreSQL acknowledges `COMMIT` or `ROLLBACK`; if cancellation drops a future mid-flight, the next serialized caller rolls back the uncertain transaction before doing new work. The public raw `PostgresGraphStore::execute` surface is deliberately autocommit-only. Its lexical guard rejects transaction-control statements in a batch while ignoring lookalike words inside strings, identifiers, dollar-quoted bodies, and comments. PostgreSQL PGQ forwards through the same contract.

Schema application adds typed label views and expression indexes. For example, a `Person` node schema with `name: String` and `age: Int` can produce a `grust_node_person` view over the universal node table, with `age` exposed as a `bigint` expression. This is a deliberately incremental typed-storage path: PostgreSQL keeps the flexible JSONB source of truth while callers that know the schema get typed SQL surfaces.

`grust-pggraph` is now the pgGraph extension layer over that same shared PostgreSQL implementation. It bootstraps the `graph` extension, registers the universal tables, and can optionally build the pgGraph projection for graph-index experiments without forking the storage and traversal code.

`grust-postgres-pgq` targets PostgreSQL 19's native SQL/PGQ support. It uses the same universal tables as the durable storage layout, creates a native `PROPERTY GRAPH` over them, and executes bounded traversal through `GRAPH_TABLE`. That keeps writes, reads, schema views, and mutation batches on the proven PostgreSQL backend while letting traversal exercise PostgreSQL's standard property-graph query engine.

The reusable SQL boundary lives in `grust-sql-core`. It owns the parts that are really common across row-store SQL graph backends: universal table DDL, reads, bounded traversal joins, mutation framing, schema views, expression indexes, identifier quoting, and literal escaping. Dialects keep the pieces that affect correctness or efficiency. PostgreSQL keeps JSONB predicates, `ON CONFLICT`, `CREATE OR REPLACE VIEW`, transaction text, and lateral joins for undirected steps. Turso keeps JSON text, `json_extract`, `json_patch`, SQLite-compatible views, and a derived-table undirected join shape. Sail does not use this SQL core because its SQL path runs through Spark Connect, Arrow IPC staging, and distributed Spark SQL rather than a direct row-store connection.

Recursive walk pushdown no longer stores raw node IDs between sentinel delimiters. PostgreSQL, Spark SQL, and the generic SQLite dialect encode IDs as hexadecimal, delimiter-free tokens before constructing visited sets. A dialect without both recursive-CTE support and an encoding hook declines variable-path or shortest-walk pushdown and preserves correctness through fallback.

`GraphSqlDialect::max_identifier_bytes` lets a dialect declare a limit for generated schema identifiers. PostgreSQL reports its 63-byte ceiling: typed node/edge view and property-index names at the limit remain valid, while longer names fail with `GrustError::Schema` before the server can silently truncate them into an ambiguous or colliding identifier. Dialects that retain the default `None` keep their existing behavior.

8.5 Turso

`grust-turso` uses the Turso Rust SDK directly. The default connection path opens a local in-process Turso database:

```
# use grust::prelude::*;
# async fn open() -> grust::Result<()> {
let store = TursoGraphStore::connect(TursoConfig {
    path: "data/grust.db".to_string(),
    table_prefix: "grust".to_string(),
    batch_size: 500,
    journal_mode: TursoJournalMode::Wal,
})
.await?;
# Ok(())
# }
```

The storage layout mirrors the universal table shape used by the PostgreSQL backend, but uses SQLite-compatible types and JSON text properties:

```
grust_nodes(id text primary key, label text not null, props text not null)
grust_edges(id text, from_id text not null, to_id text not null,
            label text not null, props text not null)
```

Reads and traversal use ordinary SQL over the local Turso connection. Schema application creates label-specific SQL views and expression indexes using `json_extract`. Mutation batches are wrapped in a Turso transaction.

The `journal_mode` option selects the concurrency model. The default `Wal` is Turso's single-writer write-ahead log. Selecting `Mvcc` enables Turso's multi-version concurrency control (`PRAGMA journal_mode = mvcc`); data writes then run inside `BEGIN CONCURRENT` transactions with bounded conflict retry, so concurrent writers make progress. Turso 0.7.2 converts a live database between the two modes in either direction. MVCC loads run over several writer connections (`set_mvcc_load_parallelism`) or fill through WAL and switch back (`set_bulk_load_via_wal`); every load path runs with foreign keys off; and concurrent single-statement writes can share one durable commit (`with_group_commit`). The chapter "Turso under strain" covers each of these, what they measured against WAL and against Neo4j, and how the advice changes on Turso 0.8.

With the `turso-sync` facade feature, callers can construct a synced store from a local path, remote URL, and optional auth token. The `GraphStore` API continues to operate on the local SQL connection; callers decide when to call the store's `push` and `pull` helpers.

8.6 QueryGraph Memory: TypeSec over Grust

`querygraph-memory` is a concrete application of the Grust backend contract. It implements TypeSec's synchronous `MemoryStore` interface over a compatible Grust mutation backend without moving authority or plaintext handling into the storage layer:

```
pub struct GraphStoreMemoryStore<G: GraphMutationStore> { /* ... */ }

use querygraph_memory::TursoMemoryStore;
```

```
let store = TursoMemoryStore::open("data/querygraph-memory.db")?;
```

`GraphStoreMemoryStore<G>` is the generic adapter for an already-initialized `GraphMutationStore`. The production-oriented v1 alias, `TursoMemoryStore`, binds that adapter to `TursoGraphStore`. `TursoMemoryStore::open(path)` creates missing parent directories, opens a file-backed local database with the stable `querygraph_memory` table prefix, and runs Grust's bootstrap before returning. `open_with_config` is the explicit form for applications that need another table prefix, batch size, path, or journal mode. A default `TursoConfig` still uses `:memory:`, so durable callers must supply a file path.

The adapter projects memory into a small property graph:

```
(:MemoryRecord {record: <opaque JSON>, space: ...})
    -[:MENTIONS]->(:MemoryEntity {name, kind})
(:MemoryEntity)-[:RELATES]->(:MemoryRelation {rel, fact_id})
    -[:RELATES]->(:MemoryEntity)
```

Each `MemoryRelation` is an assertion, not merely an endpoint pair. Its node ID is a SHA-256 identity over length-prefixed source, relationship name, target, and record ID components. Replaying one record is stable, while two records that assert the same named relation between the same entities retain separate lineage. Neighborhood traversal follows the two-edge assertion shape and also reads the legacy direct `RELATES {rel, fact_id}` representation so existing durable stores remain compatible. Tombstone preflight first discovers the record's assertion nodes and any legacy fact edges, then submits deletion of that discovered set with the record node as one mutation batch. A transactional backend makes that deletion batch atomic, and other records' assertions survive. The discovery reads are not isolated inside that transaction: callers must synchronize a tombstone with concurrent links for the same record so a new assertion cannot appear after discovery and escape the deletion batch.

The complete TypeSec `StoredRecord` is serialized into one opaque JSON property and round-tripped whole. Grust does not open the protected content. TypeSec's `MemoryVault` remains the only component that rehydrates it, verifies typed capabilities, applies a recall clearance ceiling, excludes quarantined records, joins sensitivity labels during consolidation, and emits audit evidence. That separation is the security boundary: Grust supplies durable graph mechanics; TypeSec decides which subject may learn what.

Queries push only memory-space equality into the graph as `Start::NodesByProperty`. The other `StoreQuery` dimensions—kind, time, label, entities, text, invalidation state, and ordering—continue through TypeSec's shared `StoreQuery::matches` implementation. This is semantically complete and conformance-tested, but it is deliberately not advertised as full GQL pushdown. A neighborhood traversal may discover record identifiers associated with global entity nodes across several spaces; the vault rejects every record outside the authorized space before content can be revealed. Tenant isolation in v1 is therefore authorization at the vault boundary, not physical graph partitioning.

Consolidation uses the mutation boundary rather than a special memory-only transaction API. `MemoryStore::apply_batch` converts its puts and invalidations into one `GraphMutation` slice and calls `apply_mutations` once. Turso reports transactional mutation atomicity, so superseding old records and inserting the replacement commits as one unit. TypeSec performs the SecLib label join before that batch reaches storage, which means a replacement derived from a Sensitive source remains Sensitive after closing and reopening the database. A generic backend that reports `OrderedNonAtomic` remains usable for simpler operations but cannot promise atomic consolidation.

The adapter also owns the sanctioned bridge between TypeSec's synchronous `MemoryStore` and Grust's asynchronous `GraphStore`. A dedicated current-thread Tokio runtime carries I/O and time drivers. Calls made outside Tokio drive it directly; calls made from an existing Tokio runtime execute on a scoped thread. This keeps an MCP or HTTP service from nesting runtimes or panicking, and lets the store shut down safely when dropped from asynchronous application code.

Semantic ranking and cognition preserve the same boundary. The reference `VectorIndex<E: Embedder>` is an in-process cosine index. An `Embedder` declares whether it is local; a remote embedder is never handed Sensitive or Secret text, and those records remain available to ordinary authorized recall without participating in remote vector ranking. Search returns candidate IDs, with an optional bounded entity co-mention boost, but the vault still performs the authorization and label checks. Reference deduplication, contradiction, and importance analyzers likewise make no writes. They consume already-recalled views and emit inert `ConsolidationPlan` values that must return through the capability-gated vault.

The governed cognition path extends that rule instead of bypassing it. A `CognitionRequest` contains one TypeSec-authorized input, its canonical binding, its optional vault-verified governed source scope, the exact LakeCat snapshot and policy-narrowed projection, the field mapping used by governed ingestion to derive the selected memory records, a durable job identity, and either the deduplicate or reconcile operation. The trusted host selects a fixed reference or native Sail profile before protected input is loaded; public engine implementations cannot report their own trusted identity. Both asynchronous engines return a bound `CognitionProposal`, never a store handle or direct write.

Every bound proposal uses TypeSec proposal schema version 4. That wire contract identifies `input_snapshot` with the canonical immutable snapshot digest, keeps the LakeCat grant digest as separate binding evidence, and carries an explicit `mutated` or `no_change` effect. Reference and Sail derive that effect from the complete proposal: zero drafts and zero plan steps means no-change; every nonempty plan is mutating. Earlier bound schema versions fail closed; unbound schema version 1 remains only an inert local planning value. This wire version is independent of the operation algorithm version below.

Deduplicate and reconcile each own an explicit version-2 semantic contract. Reference and Sail profiles bind the same per-operation version because they must produce the same canonical plan. Crate, package, and build versions remain useful implementation metadata, but never substitute for the algorithm version in signed TypeDID authority. Previously signed version-1 or package-

bound intents are deliberately incompatible and must be authorized again; no native profile silently upgrades their authority.

Live Sail does not re-read LakeCat rows or reinterpret the ingestion mapping. It derives and stages only authorized IDs, normalized text keys, contradiction prefix/tail keys, and validity timestamps under a collision-safe session view; it does not stage the raw text column. Planning has independent finite operation, abort, and cleanup deadlines, and cleanup is attempted after success, failure, timeout, or caller cancellation. The Spark client and cognition decoder reuse the same public 16 MiB Arrow IPC payload limit; the decoded protobuf message has one additional MiB of bounded envelope headroom. Normalized and contradiction keys remain content-derived rather than anonymized, so the Sail endpoint belongs inside the processing boundary authorized for that protected input. Arrow framing, schemas, declared rows, buffers, compressed expansion, result counts, and local reconciliation work are checked before Arrow result-array allocation and then rechecked against the complete result. The reference and Sail engines use the same deterministic planning functions, so input permutation and timestamp ties produce canonical output.

Durability is a separate storage capability. `GraphCommitStore` combines exact node or absence expectations, a mutation batch, an idempotency digest, and a backend receipt in one transaction. Turso mints the receipt's canonical UTC RFC 3339 timestamp at nanosecond precision immediately before inserting that receipt into the same transaction. It is backend-issued transaction-boundary evidence, not a wall-clock observation taken after storage fsync, and recovery returns those exact persisted bytes or fails closed on malformed time. The cognition scheduler stores only scoped digests, issues bounded renewable bearer leases, persists only the canonical proposal digest, and survives reopen. During application, TypeSec supplies the opaque prepared commit; Grust atomically checks source revisions, applies its exact memory operations and ID-only index outbox, writes the audit record, persists the outcome, and completes the job. A no-change token has empty operations, affected IDs, and outbox, and retains the prior memory version, but the exact source guards, job, audit, outcome, and guarded ledger still commit as one decision. Recovery is read-only and cross-validates the job, audit, outcome, authority scope, optional governed source scope, proposal, effect, and backend receipt. Durable outcome schema version 3 requires TypeSec audit schema version 2, which carries the same typed effect, distinct grant and snapshot digests, and the trusted authority-revalidation and preparation times. Audit and commit-envelope digests use version-3 domains so this evidence layout cannot be confused with either predecessor. TypeSec owns scope selection and authoritative reload checks; Grust preserves that evidence and atomically enforces the prepared full-record preconditions. Tests exercise concurrent identical decisions and commit-then-response loss and prove that retry plus reopen retain exactly one job, audit record, outcome, guarded ledger, and the exact mutation/outbox shape required by the effect.

The scheduler's `transitionedAt` is a caller-supplied logical transition time; for `Completed` it is explicitly the TypeSec audit's `preparedAt`. It is not a backend commit timestamp. A completed job's `completionDigest` is exactly the canonical TypeSec prepared digest for either effect, never the resulting memory version; this avoids collapsing no-change into an unchanged memory version. Recovery checks durable schema versions before deserialization, rejects incompatible historical

outcome and audit layouts, and requires affected IDs to retain TypeSec's strict canonical order. Authoritative `committedAt` evidence exists only in the outcome and receipt, must be canonical RFC 3339, and cannot predate preparation. Grust rejects malformed or regressive backend time on initial return and recovery instead of substituting a timestamp from another phase.

The scheduler and outbox APIs are storage primitives behind Marciana's authenticated scheduler and trusted worker pool. A worker intentionally need not equal the submitter, which lets expired work move safely, but canonical owner strings and scoped job keys are not credentials. Once issued, lease and claim tokens are bearer credentials for worker transitions. Marciana must authorize acquisition and cancellation and keep those tokens confidential; only a freshly prepared opaque TypeSec commit can authorize memory mutation.

This is a native Grust, TypeSec, LakeCat, and Sail composition. Cognee supplied design inspiration only; no Cognee runtime, adapter, or storage dependency is present.

The durable proof still has explicit limits. `MemoryId::next()` uses a process-local counter, so a restarted ordinary writer can collide with persisted `mem-N` identifiers; a hosted or multi-process service should mint collision-resistant IDs at its boundary. `VectorIndex` is not a persistent LanceDB ANN implementation, and memory predicates beyond space are not fully pushed into GQL. Cognition job idempotency does not replace durable TypeDID nonce replay protection shared across gateway replicas. Finally, vault-level tenant checks, restart persistence, and running-service conformance do not by themselves constitute a hosted multi-tenant service with quotas, migrations, backups, deletion propagation, and service-level objectives.

8.7 Sail

`grust-sail` connects to a Sail Spark Connect server over gRPC. It stores graph data in Spark DataFrames backed by Delta tables. SQL commands and reads are sent as Spark Connect SQL relation plans, and read results are decoded from Arrow IPC streams.

Portable count projections may lower to `SELECT 1` when no named binding must be returned. Sail sends that row-presence marker as an Arrow integer rather than a string. Grust decodes integer markers alongside text columns, preserving one row per match for the shared Rust projection; it does not mistake this path for a server-side aggregate. This boundary is regression-tested for multiple result batches, nulls, and empty results. Sail 0.7.1 cannot execute the generated recursive walk CTE, so variable-length path reads use the shared reference fallback. Benchmark evidence labels this as graph materialization plus Rust execution, not native Sail path performance; downloaded-scale admission may reject that materialization path.

Call `SailGraphStore::close().await` after a session's operations finish to release its remote temporary views and session state. This consumes the Rust client and invalidates any other client sharing that session ID; durable warehouse files are not deleted. Ordinary Rust drop cannot perform this asynchronous cleanup. Bound the close future if your application requires a deadline, and treat timeout or failure as uncertain cleanup. A release acknowledgement alone does not prove an interrupted query has stopped.

Connection establishment validates the client configuration. The default `SailWarehouse::ServerManaged` policy does not set `spark.sql.warehouse.dir`; Sail's catalog and warehouse configuration remain authoritative. Connection failures use a stable message and do not render the configured endpoint or transport error because either can disclose endpoint credentials or signed parameters. The warehouse policy is safe across a remote client boundary and does not silently select a new client-local persistence path. Co-located development can opt into `SailWarehouse::LocalSessionScoped`, which derives a path beneath the client's temporary directory from the session ID. Grust does not delete that directory; callers own cleanup, and reusing the session ID reuses the path. Durable callers can use `SailWarehouse::ExplicitPath` with a stable absolute path visible to the server; Grust sets and reads that override back through the same Spark Connect session. Reopening tables across sessions additionally requires Sail to provide persistent catalog metadata. If Sail's unconfigured warehouse fallback is the relative `spark-warehouse` path, the server needs an absolute setting or the client must select one of the explicit Grust overrides before creating managed Delta tables.

Bulk writes stage Arrow IPC batches as Spark Connect `LocalRelation` temp views and then merge from those views. That avoids building one giant SQL literal per row, keeps user values out of SQL text, and gives long-running requests an operation id with reattachment enabled. Query filters bind user values through Spark Connect named arguments. Delete mutations stage their values as Arrow temp views before running argument-free SQL commands, which avoids string substitution while matching Sail's current command-parameter behavior. Traversal joins use globally unique node ids; source and destination label columns may be empty for single-edge writes where the full graph is not in scope.

Sail's Arrow boundary is now public too. Applications can stage arbitrary Arrow IPC streams as session temp views and query them with Spark SQL, collect Spark SQL results as Arrow IPC chunks, or load Grust-shaped node and edge IPC streams through the normal graph write path. This gives Sail the same data-source role as Ladybug while keeping Sail's internal Arrow 58 dependency separate from Ladybug's Arrow 55 dependency. Staged views can be dropped through a validated, idempotent helper, allowing protected batch inputs to be cleaned up on success, execution failure, or an uncertain retry.

When a schema is applied, Sail creates typed Delta tables per node and edge label and mirrors writes into them with `MERGE INTO`. The universal Spark tables keep traversal simple and portable; the typed tables make declared graph labels available as ordinary Spark columns. Their declared names survive table creation, while Delta constraints reject null structural node and edge identities. Constraint-registry values likewise enter through staged Arrow rather than SQL string interpolation.

Sail also has reusable graph analytics helpers over the persisted generic tables. `read_graph` collects the generic `grust_nodes` and `grust_edges` tables back into a portable Grust `Graph`. `in_degrees`, `out_degrees`, `degrees`, and `degree_pairs` run Spark SQL over those same tables and decode the Arrow results into small Rust row types. These helpers are deliberately low-level: they expose common graph measurements without making the backend-neutral `GraphStore` trait depend on Spark-specific analytics.

The Sail crate now also publishes its generic table contract. Constants name `grust_nodes`, `grust_edges`, and the physical node and edge columns, including the persisted generic edge `edge_key` and optional explicit edge `id`. Projection helpers classify requested graph fields as physical columns or JSON properties, map edge `label` to the stored `edge_type`, and check when a typed Sail node or edge table can satisfy a graph query directly. This keeps Sail-native graph planning and GrustFrames-style distributed lowerings aligned with the same backend layout that Grust writes.

The same contract layer exposes typed table descriptors derived from `GraphSchema` and directional triplet SQL. The triplet helpers join generic edges back to their source and destination nodes, and can orient rows as outgoing, incoming, or undirected pairs. That is the shared primitive needed by distributed triplet filters, motif expansion, and aggregate-message style passes without making the backend-neutral store trait depend on those higher level algorithms.

Writable Cypher is not a separate Sail persistence path. The portable Cypher/GQL layer (see *Cypher and GQL*) owns parsing, semantics, planning, and the read engine; for the bounded read subset it lowers the `MATCH`/`WHERE` filter into Spark SQL while the `RETURN` projection runs through the shared reference. `SailGraphStore` executes resolved `GraphMutationPlans` through `CypherMutationExecutor`, so Cypher writes use the same staged-Arrow, `MERGE INTO`, typed-Delta mirror, and delete paths as ordinary Grust writes. The plan and report types are backend-neutral — Memory, Sail, and Turso all execute them — while Sail adds the SQL read pushdown and typed-table mirrors on top.

8.8 FalkorDB, HelixDB, and SurrealDB

The FalkorDB backend writes through Redis `GRAPH.QUERY` using Cypher-like `MERGE` statements. It batches nodes by label path and edges by relationship type. Schema application creates label/property indexes for declared node types. Configurable identity-property names and generated labels, relationships, and properties are validated before Cypher construction. Property names retain their physical spelling through backtick quoting where FalkorDB permits it; unsafe delimiters and normalized-name collisions within a schema or complete graph load fail closed. The configured structural ID wins over property-map data, and connection-pool/query failures do not render the Redis URL or credentials.

The HelixDB backend has HTTP and SDK stores. Both support batched writes, node reads, edge reads, and backend-neutral traversal through Helix dynamic queries. Edge writes store Grust relationship metadata (`relationship`, `from_id`, `to_id`, and optional `edge_id`) so `EdgeQuery` can reconstruct Grust edges from Helix relationship rows. Helix writes preserve supported scalar and array properties instead of silently dropping non-string values; unsupported JSON object properties return an explicit error. The current schema hook validates that labels, relationships, and fields can be safely lowered through the dynamic-query path; backend-native schema-file generation can build on that same `GraphSchema` contract later. Schema and graph preflight also reject normalized relationship collisions and attempts to declare or write the structural `id`/label and edge-metadata fields. Both HTTP and SDK graph loads validate all chunks before transport, and transport failures omit configured URLs and their embedded credentials or query strings.

The SurrealDB backend also has HTTP and SDK stores. It can bootstrap, clear, upsert nodes, relate edges, delete nodes and edges, read nodes and edges, and execute traversal hop-by-hop through the `GraphStore` contract. Applying a schema lowers node and edge declarations to Surreal `DEFINE TABLE` and `DEFINE FIELD` statements so the backend can run in schemafull mode where the schema calls for it. Reads use configured labels and relationships, plus ID-derived table names, to find records across Surreal's label-specific tables. Generic edge reads and node deletes require `SurrealConfig.relationships`; when that list is empty, the backend returns a configuration error instead of silently scanning no relation tables. Explicit edge-label reads and deletes can still target a known relation table directly. Traversal batches target-node reads per step through `get_nodes`, avoiding a serial node lookup for every edge in a fan-out. Mutation batches are wrapped in SurrealDB transactions. Both transports push edge endpoint filters to SurrealQL using `meta::id(in)` and `meta::id(out)` with complete escaped logical IDs, then retain the Rust postfilter. This avoids transferring unrelated relation rows without guessing an endpoint table from its ID prefix. It is not a claim that the server uses an index seek; live query-plan and performance validation remain separate work. SurrealDB 3.2 identifiers are quoted without lossy property-name rewriting. Every newly written node stores its original, case-sensitive Grust label in the reserved `__grust_label` field instead of reconstructing that label from a normalized, lowercase physical table. Reads of older rows fall back to the physical table label exposed as `__grust_physical_label`. Record decoding separates the table at the first colon and removes only a matching outer backtick pair, preserving colon-bearing logical IDs such as `City:4` without a trailing backtick. Configuration, schema fields, normalized node/relation table claims, reserved storage fields, and complete graph batches validate before bootstrap or write I/O. Optional Grust edge IDs are stored separately as `edge_id`; node `id` and `labels`, relation `in`/`out`, and internal metadata cannot be overwritten by user properties. HTTP and WebSocket failures omit URL userinfo and query secrets.

8.9 CocoIndex

`grust-cocoindex` is intentionally not an ordinary `GraphStore`. CocoIndex is an incremental target-state system, so the Grust integration exports a graph as serializable node and relationship state:

```
let export = graph.to_cocoindex_export()?;
let graph = cocoindex_export_to_graph(export)?;
```

The export uses Grust node IDs as target keys, converts properties to JSON, and requires edge endpoints to exist in the graph so relationship source and target labels can be emitted. The import path expects the same key shape, validates relationship endpoints against imported nodes, and preserves explicit relationship keys as Grust edge IDs.

8.10 Backend Integration Tests

Unit tests stay self-contained, but live backend tests are explicit. They are marked ignored in Cargo so a normal workspace test run does not accidentally depend on local services. When a live test is requested, however, it must reach the backend; it no longer returns early and pretends success when a server is missing.

The repository provides a launcher for those checks:

```
scripts/integration-test.sh doctor --profile docker --mode docker
scripts/integration-test.sh --profile docker --mode docker
```

The Docker profile is the contributor path: it starts the Docker-backed services and runs local LanceDB and CocoIndex checks. The full maintainer profile is still available:

```
scripts/integration-test.sh --profile all
```

The launcher reads `integration/backends.conf`. In `auto` mode it prefers already-running services, then configured local source checkouts such as `~/src/sail`, `~/src/SurrealDB`, `~/src/FalkorDB`, and `~/src/HelixDB`, then Docker Compose where a service is available. The repository-level `docs/INTEGRATION.md` guide covers profiles, modes, Docker image pins, source-checkout configuration, and CI strategy.

Acorn 0.14.0 is a lockstep release of all publishable Grust crates. It adds reusable graph algorithms, optional Arrow interchange/results, extensible Cypher procedures and shared resource admission. It retains earlier Sail session and path-admission fixes and the backend identity/transport corrections documented in the repository changelog. Backend-native feature parity remains explicit; the portable local algorithm executor does not imply native backend analytics.

Surreal and Helix HTTP distinguish bulk-load batch size from incremental writes. LanceDB reuses table handles while checking latest-read consistency; an async gate prevents a concurrent first lookup from republishing a handle across local table recreation. Turso's sync constructor initializes its snapshot cache, and a pull retires cached captures before it can change local data, even if cancelled.

LanceDB stays at 0.30.0: the attempted 0.38.0 default-feature local build fails within upstream `lancedb` because `job.rs` references the remote-only `Error::Http` variant when `remote` is disabled. The unpublished Helix adapter now targets exact `helix-db` 3.0.0: its SDK path uses typed nested-AST `QueryRequest` builders and `Client::query(request)` rather than `DynamicQueryRequest` and `dynamic_query`. The direct HTTP/v1 store remains separate, with explicit bulk-load batching. Unit tests cover SDK serialization and HTTP behavior. The separate SDK/v2 Docker example run now has 264 passing observations across baseline and adversarial queries, with a retained runtime audit. Historical HTTP service evidence is not reused as proof of SDK/v2 compatibility. The repository's `benchmarks/lsqb/BACKENDS.md` records the full qualification matrix and live gate evidence.

8.11 LSQB Compatibility Microbenchmark

The repository also carries a Docker-reproducible compatibility workload in `benchmarks/lsqb`. The unmodified upstream side pins Graph Data Council LSQB commit `242cb2fd31340ca688954cb94794d74c0d5b6f92`, LadybugDB 0.19.0, and a digest-pinned Python 3.12.11 container. It validates LSQB's nine count queries independently of Grust.

The adapted Grust side is rectangular across twelve declared backends: Memory, Turso, PostgreSQL, Ladybug, FalkorDB, SurrealDB, LanceDB, Sail, pgGraph, PostgreSQL PGQ, Helix, and CocoIndex. Baseline and adversarial suites each contain one cell per backend, yielding 24 count

reports in a complete run even when a declared cell is unsupported, unavailable, or not applicable. For every backend, the baseline cell declares nine LSQB-derived queries and the adversarial cell declares 13 separately labeled count attacks. A distinct backend-neutral suite has 14 required policy rejections, so the adversari.al extension contains 27 attacks across two non-overlapping expectation models.

The executable additionally exposes distinct `helix-sdk` and `surreal-sdk` network-client lanes, while the historical twelve-backend launcher and receipts remain unchanged. Both Helix and Surreal retain their direct HTTP lanes. Surreal's separately published Rust SDK/WebSocket example cohort passes 108 baseline and 156 adversarial observations, with two warm-ups and ten measured repetitions per query in rotating order. It measures backend materialization plus Rust reference execution, not native Surreal query-engine performance. Load, worker setup, query, and recovery timings remain separate. Helix's SDK3 source-built `/v2/query` service also passes all 264 example observations under the same sampling protocol. Both SDK cohorts are independently admitted on adversari.al/graph. Helix's frozen bundles retain client and server build receipts, recipes and logs alongside the observations and runtime records. Neither a build guide nor a matching image label alone establishes a result.

Reports name the measured execution class as `in-process-reference`, `backend-native-aggregate`, `backend-row-source-rust-projection`, `backend-materialize-rust-reference`, or `backend-neutral-policy`. Each query ends as `pass`, `mismatch`, `unsupported`, `unavailable`, `timeout`, `error`, or `not_applicable`; policy cases separately record pass/fail and a stable rejection category. A capability or service gap remains visible rather than becoming a fallback pass. Those execution classes are not performance equivalent and their timings must not be collapsed.

New matrix observations also record a worker-declared execution `plan`: `clause-pipeline`, `count-factorized`, `sql-row-source`, `sql-count`, or `backend-native`. This is the selected Grust route, not the database's physical plan. It is retained in incremental records, including timeouts. Historical observations without it remain explicitly unknown; frozen Neo4j and upstream bundles are not rewritten. Mixed plans do not form one timing distribution.

The 28-node, 72-edge `sfexample` graph is a conformance and orchestration gate, not a backend ranking. Authenticated LSQB SF0.1 and SF0.3 downloads add larger tiers with sealed archive and extracted-manifest identities. At those scales, the harness admits in-process reference, backend row-source with disclosed Rust projection, and backend-native aggregate execution, while whole-store materialization plus the Rust reference is explicitly `unsupported`. The manifest additionally binds a per-query, per-execution-class logical-row bound: only exact cardinalities or certified upper bounds at or below 1,000,000 are timed on downloaded tiers for row-producing Rust plans. Larger or insufficient Rust-row bounds are explicit unsupported outcomes without samples. A separate, hash-bound registry admits proven indexed counts as `count-factorized` with `not-materialized` Rust rows; this describes the algorithm, not an empty query result or zero memory usage. Native scalar aggregates remain a separate class. For the pinned workload, Memory selects indexed counts for all 22 cases (q1–q9 and a1–a13); Turso and PostgreSQL select scalar SQL counts for q1, q4, a1 and a7. Unproven shapes retain their existing routes and gates. All 22 example cases pass offline on Memory and embedded Turso, but these checks do not qualify a performance run

or a live PostgreSQL service. Index construction is load work; parsing, semantic analysis, planning and execution remain query-timed. The fourteen-case policy suite remains fixed to `sfexample`. Sail, PostgreSQL PGQ, and Helix have no default pinned service startup contract; their cells are `unavailable` unless an operator explicitly qualifies a digest-pinned, resource-limited external service.

This is a conformance and reproducibility microbenchmark, and LSQB is not an official LDBC benchmark. A complete Grust matrix requires a valid publication receipt; separate native Neo4j, source-built Sail, Helix SDK and Surreal SDK cohorts use immutable evidence bundles admitted by an independent site verifier. A bundle manifest's hashes alone do not establish qualification. Unadmitted diagnostic and discovery runs are not publication evidence. The matrix receipt inventories one normalized watchdog record per cell, binding the configured hard limit and elapsed wall time to the child exit status and exact container ID, name, project, and service observed by the supervisor. Missing, timed-out, or cross-cell records are rejected.

These are not LDBC Benchmark Results.

The evidence home and canonical public presentation are at adversari.al/graph.

9 Cypher and GQL

The property graph model so far is a Rust API: builders, traversals, and the store contract. `grust-cypher` adds a query and mutation *language* on top of it — a backend-neutral GQL/Cypher layer — without changing that core.

The language is built as a real pipeline, not ad-hoc string handling: a span-bearing lexer, a recursive-descent parser into a typed AST, and a semantic analysis pass that resolves bindings and kinds. A conformance spine — a `GqlFeature` taxonomy with structured errors — records exactly which constructs are supported, planned, or out of profile, so the surface is auditable rather than aspirational.

9.1 A portable read core

Reads run through a Memory *reference executor* over a graph snapshot: `MATCH` and `OPTIONAL MATCH` (with null padding), multi-hop and variable-length paths, a three-valued `WHERE` expression engine, and `RETURN` with aliases, `DISTINCT`, `ORDER BY`/`SKIP`/`LIMIT`, aggregates with implicit `GROUP BY`, `WITH`, `UNWIND`, and `UNION`. Backend execution is checked against this shared executor and independent semantic fixtures.

Numeric `sum()` excludes null values and returns integer zero when no numeric values remain, including an empty match or a null-only group. `avg()` returns null in those cases. An ungrouped empty aggregate produces one result row; a grouped query with no input produces no groups. The same rules apply to DISTINCT sums and the streaming aggregate path. These individual contracts do not imply complete Cypher compatibility for every aggregate type or function.

Lists have three binding forms, usable anywhere an expression is: a fold `reduce(acc = seed, item IN list | body)`, a comprehension `[item IN list WHERE predicate | projection]` with either clause optional, and the quantifiers `any`/`all`/`none`/`single(item IN list WHERE predicate)`

over arbitrary lists and predicates. Each form pushes an immutable lexical scope over the row rather than modifying it, so inner forms see outer bindings and `sum(reduce(...))` folds per row inside an aggregate. A binding name that already exists in the row is rejected during semantic analysis, not resolved by precedence. An empty list folds to its seed, a `NULL` list yields `NULL`, a `NULL` element is handed to the body, and quantifiers follow three-valued logic. Every element charges one work unit, so a budget, cancellation, or deadline stops a long fold between elements. No dialect lowering exists yet: every pushdown planner declines a query containing a binding form and the store answers from the reference executor. A write statement's `RETURN` evaluates the same forms through the same evaluator over its materialized bindings; the earlier restricted write shape `item IN variable.property WHERE item = value` keeps its exact-equality results.

Candidate rows share immutable node and edge bindings as patterns expand. Over an owned graph a matched node or relationship is bound by reference into that graph, which outlives every row of the query; an element that a typed index builds on demand is moved into one shared binding. Variable-length trails, shortest-path reconstruction and path accumulators hold elements the same way, so copying a candidate or a trail never clones a property map. A row itself is a small key-sorted vector of bindings. Projected results remain owned, and edge-slot identity and row order remain unchanged. Logical full-element copy charges remain conservative even when the physical representation shares storage. This reduces repeated allocation; it does not make the complete MATCH pipeline streaming or impose a whole-process memory cap.

The read core also composes: `CALL { … }` subqueries execute once per incoming row with the outer bindings visible (correlated import-all scoping) and join their `RETURN` columns back onto the row, and `shortestPath(…)` / `allShortestPaths(…)` find minimal-length simple paths per endpoint pair over a relationship segment. Procedures generalize to table-valued functions: `CALL name(args) [YIELD …]` evaluates its arguments against each incoming row (`tvf.range`, `tvf.keys` join the `db.*` catalog procedures).

Backends that can materialize SQL push a supported read subset down: a query's `MATCH`/`WHERE` filter lowers into backend SQL, while the `RETURN` projection normally runs through the shared reference. Turso and PostgreSQL additionally opt into a conservative scalar `COUNT(*)` lowering over existing match sources, returning one integer to Rust for decoding and final pagination. Sail has not opted into scalar aggregation. Segment and multi-pattern SQL joins decline overlapping relationship types when physical edge independence is unproven; nullable or duplicate edge IDs are not a valid uniqueness guard. Scalar filters admit only conjunctions of genuine property/string equality with exact JSON payload-type checks and byte-wise comparison, in addition to structural node labels. Numeric and ambiguous inline-label filters keep their older routes. Existing row-source SQL filter/coercion restrictions still apply; an embedded-SQLite differential oracle checks supported reference-vs-pushdown results. Sharing the final projection alone does not prove every backend SQL predicate equivalent.

`grust_cypher::read::run_read_query_indexed` uses a reusable `TypedGraphIndex` and selects exact count algebra for proven query shapes. Forests require globally distinct relationship types and allow labels and inline scalar literal properties. The wedge supports labels but no inline property maps: two undirected hops with unequal outer endpoints are weighted by a differently

typed outgoing leaf. Parallel edges, reciprocal edges, self-loops, empty-match zero and final scalar pagination are preserved. General predicates, grouping, unproven repeated types and other unsupported shapes use the existing reference executor. A forest may also carry independent single-edge optional leaves; each contributes its number of matching edges, or one padded row if none match. Optional anchor predicates stay optional. A narrowly supported plain `WITH` before those leaves preserves multiplicity even when it drops bindings; filters, nullable anchors and dependent optional branches retain fallback. Mandatory forest branch combination reuses borrowed necessary candidates from predicate evaluation. Outside-seed weights remain zero; neither optional padding nor child products revive them. Full weight-array initialization, optional scans and root sums retain their existing accounting. Mandatory child-branch scans prepay borrowed chunks of at most 256 physical adjacency slots, including incoming self-loop copies that are skipped when counting. Successful work totals remain exact and no storage is added; a tight budget can reject a whole chunk before its affordable prefix. Per-predicate charges and checkpoints remain, while property-free scans checkpoint at most 256 slots apart. Optional execution is unchanged. Property-bearing roles with at least two mandatory incident atoms first require a nonempty typed adjacency row for each atom in its role-relative direction. This necessary-condition prefilter resolves each type once into a borrowed `TypedAdjacencyView`, retaining a charged per-role vector of views and directions. Each actual row probe is still charged; it does not scan neighbors or allocate candidates. Enabled roles may choose a strictly shorter borrowed sparse source list for a mandatory directed atom instead of their label/full-domain seed, charging one metadata inspection per atom. Ties retain the current seed; undirected atoms and dense storage do not supply source lists. The view's `sparse_outgoing_sources()` and `sparse_incoming_sources()` distinguish dense storage (`None`) from an absent relationship (`Some(&[])`). Source slots are sorted and unique, without copying or intersecting lists; every original predicate and required-row check still applies to survivors. Disabled or empty-candidate roles prepare nothing. Views borrow the immutable index and retain its dense/sparse and absent-row behavior without cloning the snapshot. Original predicates remain for survivors; optional edges never qualify a mandatory role. Degree-one and property-free roles stay unchanged. This structural heuristic can add overhead on dense roles; it is not a general selectivity estimate or a measured speedup guarantee.

Weighted tag intersections and optional-null tag/wedge anti-joins preserve the distinction between witness existence and witness multiplicity. Wedge anti-joins subtract weighted support triangles; they share a degree-oriented topology helper with location triangles and account for their additional scratch space. Support edges store checked compact multiplicities and widen before count arithmetic. The helper orders forward targets by explicit `(simple support degree, stable graph vertex slot)` rank, not multiplicity or original ordinal. For x–y, y's forward neighbors all rank above y, so the intersection safely starts strictly after y in x's sorted row. Callbacks map the three ranks back to original active-domain ordinals; location-triangle and wedge anti-join role weights and semantics are unchanged.

Constructing that rank order costs O(V log V) and charges two V-entry `u32` maps, ordinal-to-rank and rank-to-ordinal, retaining only the latter after forward adjacency is filled. Here V is the active domain size and M the number of distinct non-loop support pairs. The helper's scratch remains O(V + M); forward-row sorting costs O(M log M) in the worst case and triangle intersec-

tions remain O(M^(3/2)). These structural bounds are not a measured speedup claim. Fixed-cost anti-wedge mask scans precharge small chunks, retaining cumulative work totals and end-of-pass deadline checkpoints. Wedge role masks seed unconditional roles during initialization and use the label index for the others, preserving every label condition and overlapping roles. Leaf and non-anti center traversal reuse those borrowed candidates with full mask checks and per-candidate charging. Unlabeled roles still scan all vertices, and full-size mask/leaf arrays and their initialization remain accounted. Non-anti wedges merge each center's neighbors once, accumulating its A degree, weighted C leaves and A/C overlap. The exact checked `u128` product minus overlap preserves the unequal-outer-node constraint and physical edge multiplicities without additional scratch. Scalar narrowing happens only after subtraction; the wedge anti-join's support algorithm is unchanged. The index's global edge bound allows checked `u32` multiplicities/degrees and checked `u64` weighted leaf totals; overlap, final products and accumulated counts stay `u128`. Multiplicities come from drained adjacency spans, while raw scan indices remain `usize` because loops occur in both adjacency rows. Non-anti physical-slot scans prepay at most 256 slots per chunk, preserving exact successful work totals and separate grouped-endpoint charges. Deadline checks remain bounded even within large parallel groups and cover the end of traversal. An insufficient budget may conservatively refuse a partly affordable chunk. Directed four-cycles narrow role candidates through required labels, retain every predicate, and choose adjacency merging or binary probes. Symmetric location triangles use sparse country/path weights and oriented intersections; neither creators nor locations are assumed functional. Scalar scans count proven nonnull node/edge bindings, zero-hop identities, bounded literal ranges, simple null/constant-string probes and compatible scalar unions without match-row buffers. Range limits, null semantics and duplicates remain part of each proof. Scalar literal predicates borrow property values; comparing an incompatible complex JSON value does not clone its payload.

The classifier and executor share their eligibility proof; a query ID never selects an optimization. The repository's `docs/INDEXED_READS.md` documents the exact API and shape boundaries.

The reference executor also enforces relationship uniqueness within a single `MATCH`, including comma paths. It uses physical edge slots, not application IDs, and starts a new uniqueness scope at the next `MATCH`. Named fixed edges keep their identity through `WITH` aliases, bare-variable grouping and `WITH DISTINCT`; internal slot keys never appear in public results. Repeated relationship-list bindings are explicitly unsupported; shortest selection and node-simple variable-length traversal retain their documented restrictions.

Applications that intentionally expose a small read-only surface can use `ReadQueryPolicy`, `validate_read_query`, and `run_bounded_read_query`. This is more than a final `LIMIT`: the parser-backed gate rejects updating and unsafe query shapes, while the in-memory reference executor enforces serialized query, parameter, graph, and output sizes; node and edge counts; cumulative candidate scan and expansion work; cumulative intermediate bytes; result rows; range allocation; cumulative path hops; and a cooperative wall-clock timeout. The intermediate budget accounts cloned bindings, expression and aggregate results, and DISTINCT/GROUP keys before a final `LIMIT`. Scalar and table-valued ranges also keep a library-wide `MAX_RANGE_ITEMS` ceiling. Correlated `CALL { ... }` subqueries charge every repeated node/adjacency index build, and catalog

procedures charge each graph scan, so an outer row cannot reset that work or its deadline. Authorization, tenant-safe graph projection, remote-backend deadlines, and process isolation remain the host's responsibility; the cooperative timeout is not an operating-system hard kill.

`run_bounded_read_query_indexed` applies that same policy to an immutable indexed snapshot, including fallback execution. Planner work, masks, count arrays and typed adjacency scans consume the active budgets. Exact serialized graph size is cached at index construction, so each query checks the same byte limit without serializing the graph again. Snapshot construction occurs before the query call and is outside its deadline and intermediate budget; the host must separately bound loading and resident memory.

9.2 Values, procedures, transactions, writes

The value model gains first-class lossless `Decimal` (SQL `DECIMAL`-style) and ISO 8601 `Duration` types alongside the temporal `DateTime`, with parsing, ordering, and checked arithmetic wired through every backend. Read-only catalog procedures (`CALL db.labels()`, `db.relationshipTypes()`, `db.propertyKeys()`) expose schema metadata, and a `START TRANSACTION`/`BEGIN`/`COMMIT`/`ROLLBACK` command surface pairs with honest per-backend atomicity capability reporting. Caller-owned DDL metadata can also be materialized as a portable `CypherCatalogSnapshot`, with deterministic metadata tables for `db.graphs`, `db.graphTypes`, `db.indexes`, and `db.constraints`. Read queries may include `USE <graph>`; the default single-graph executor accepts `USE default`, and callers can bind a graph snapshot to another explicit graph name. Standalone `USE`, `SET`, and `RESET` commands update portable `CypherSession` state without changing transaction-control behavior. Fixed-length path bindings now return first-class `Value::Path` values while preserving the existing JSON path shape, and `Value::Graph` adds first-class set-shaped graph values (deduplicated node/relationship sets built with `graph(nodes, relationships)`). For work that deliberately steps outside the portable surface, `NativeQuery` is an explicit backend-native escape hatch with per-backend language capability flags — FalkorDB accepts native Cypher, SurrealDB accepts SurrealQL, and the SQL backends accept their own dialects — with structured non-support everywhere else.

The writable subset routes acceptance through the same standards-conformant parser, keeping the mutation plans byte-identical to the established write planner while narrowing the public surface to standard Cypher. Pattern-driven writes widen to multiple relationship patterns per statement, incoming `<-[:T]-` edges, and cross-variable correlated `SET`.

9.3 A backed profile

What the layer claims to support is stated precisely in `docs/GQL_PROFILE_STATEMENT.md`: the realized profile is the set of `Supported` features in Grust's scoped manifest. The internal profile is named `Full39075`, but it is not a claim of complete ISO/IEC 39075 certification or uniform backend execution. Seventy-two of the 77 Grust-catalogued features are implemented; the other five are intentional strict-write rejections. A test pins that scoped-out set to the feature manifest, while backend descriptors and integration tests record which execution paths are reference, pushed, native, or unsupported.

10 Example: A Conference Graph

Here is a complete graph-building example:

```
use grust::prelude::*;

fn conference_graph() -> Graph {
    let mut g = Graph::builder();

    let rust = g
        .node("Conference", "conf:rust-graph-day")
        .prop("name", "Rust Graph Day")
        .prop("city", "San Francisco")
        .finish();

    let talk = g
        .node("Talk", "talk:backend-neutral-graphs")
        .prop("title", "Backend-Neutral Graphs in Rust")
        .prop("track", "systems")
        .finish();

    let ada = g
        .node("Person", "person:ada")
        .prop("name", "Ada Example")
        .prop("role", "speaker")
        .finish();

    g.edge("HAS_TALK", &rust, &talk).finish();
    g.edge("PRESENTED_BY", &talk, &ada)
        .prop("confirmed", true)
        .finish();

    g.build()
}
```

The resulting graph can be loaded into memory, exported to CocoIndex target state, or sent to a configured backend:

```
# use grust::prelude::*;
# async fn load<S: GraphStore>(store: &S, graph: Graph) -> grust::Result<()> {
let report = store.put_graph(&graph).await?;
assert_eq!(report.nodes, 3);
assert_eq!(report.edges, 2);
# Ok(())
# }
```

The same graph can be loaded through the typed backend path by declaring the schema and using `put_typed_graph`:

```
# use grust::prelude::*;
# async fn typed_load<S: GraphStore>(store: &S, graph: Graph) ->
```

```
grust::Result<()> {
let schema = GraphSchema::builder()
    .node(
        "Conference",
        vec![
            Field::required("name", FieldType::String),
            Field::required("city", FieldType::String),
        ],
    )
    .node(
        "Talk",
        vec![
            Field::required("title", FieldType::String),
            Field::required("track", FieldType::String),
        ],
    )
    .node(
        "Person",
        vec![
            Field::required("name", FieldType::String),
            Field::required("role", FieldType::String),
        ],
    )
    .edge(
        "HAS_TALK",
        vec![Label::new("Conference")],
        vec![Label::new("Talk")],
        Vec::<Field>::new(),
    )
    .edge(
        "PRESENTED_BY",
        vec![Label::new("Talk")],
        vec![Label::new("Person")],
        vec![Field::required("confirmed", FieldType::Bool)],
    )
    .build();

let report = store.put_typed_graph(&schema, &graph).await?;
assert_eq!(report.nodes, 3);
assert_eq!(report.edges, 2);
# Ok(())
# }
```

That call checks node labels, edge labels, required fields, field types, and edge endpoint labels before writing. Backends then use the same schema in their own way: memory validates, LanceDB mirrors into typed Arrow tables, Sail mirrors into typed Delta tables, pgGraph exposes typed SQL views and indexes, SurrealDB defines schemafull tables and fields, and FalkorDB creates useful indexes.

Traversing from a conference to speakers becomes a backend-neutral expression:

```rust
let speakers = store
    .traverse(
        Traversal::from_node("conf:rust-graph-day")
            .out("HAS_TALK")
            .to("Talk")
            .out("PRESENTED_BY")
            .to("Person"),
    )
    .await?;
```

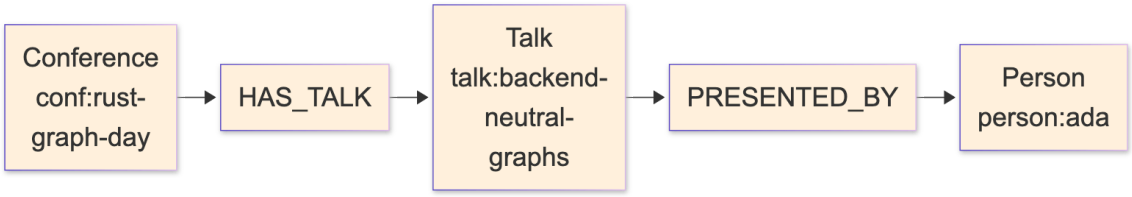


Figure 6: Diagram 6

11 Schema and Validation Direction

Grust has schema types: `GraphSchema`, `NodeType`, `EdgeType`, `Field`, `FieldType`, `EdgeUniqueness`, and `GraphConstraint`. `GraphSchema` validates labels, required fields, field value types, edge endpoint labels, edge direction, declared edge uniqueness, and required-property constraints. `FieldType` includes scalar strings, integers, floats, booleans, RFC 3339 date-times, string/int/float arrays, and JSON. Date-time values use the opaque `RfcDate` type internally; construct them through `Value::datetime` or `RfcDate::parse` so invalid strings cannot bypass validation. The default `GraphStore::apply_schema` implementation remains a no-op, which lets schemaless or schema-later backends work without ceremony.

Schema becomes more important for backends that want typed tables, indexes, or label-partitioned layouts. The current schema-capable backends use it in different ways:

- Memory stores the schema and validates local writes.
- FalkorDB creates label/property indexes for declared node types.
- Helix validates schema names that can be lowered through dynamic queries.
- LadybugDB can run in untyped dynamic mode or typed schema-applied mode; typed mode validates writes against the applied schema.
- LanceDB creates typed Arrow tables per node and edge label and mirrors writes into them.
- pgGraph/PostgreSQL/PostgreSQL PGQ exposes typed label views and expression indexes over the universal tables.
- Sail creates typed Delta tables per node and edge label and mirrors writes into them.
- SurrealDB lowers schema into `DEFINE TABLE` and `DEFINE FIELD` statements.

`apply_schema` is therefore a backend metadata hook, not a portable promise that every future write is enforced by the database. `put_typed_graph` always validates the whole graph with `GraphSchema::validate_graph` before applying schema metadata and writing. Callers

that need backend-independent guarantees can run the same validation before ordinary `put_graph` or single-element writes. Memory and LadybugDB enforce the applied schema on subsequent local writes; Sail and LanceDB validate writes before mirroring them into typed tables. FalkorDB, Helix, pgGraph, and SurrealDB currently use schema primarily for indexes, query-shape validation, views, or backend-native definitions. Constraint handling follows the same honest-capability rule. Required-property constraints validate through `GraphSchema`; unique-property constraints validate inside `GraphSchema::validate_graph`, and the memory backend reports validate-before-write behavior for them. Memory also supports explicit native constraint application through `GraphStore::apply_native_constraint`: it stores backend-owned required or unique property constraints, validates them against the current graph before accepting the request, honors `if_not_exists`, and enforces accepted constraints on later writes without requiring typed `GraphSchema` metadata. Backends that have not added a read-before-write preflight or native enforcement should continue reporting metadata-only behavior through `GraphStore::constraint_capability` and unsupported native DDL through `native_constraint_capability`. `grust-cypher` also exposes `apply_cypher_native_constraints` for applying parsed `CREATE CONSTRAINT` DDL through `GraphStore::apply_native_constraint`.

A flexible backend can keep universal node and edge tables as the portable interchange surface. A typed backend can add native tables, fields, indexes, or constraints behind the same `GraphStore` trait.

11.1 Semantic Model Projection

The public facade can also project a versioned semantic model into this same property-graph shape. `SemanticModelProjection` describes datasets, their fields and physical sources, metrics, and named relationships, bound to a positive model version and SHA-256 source-artifact identity.

`semantic_model_graph` validates nonempty normalized names, SHA-256 formats, dataset references, and per-scope uniqueness before construction. It uses length-prefixed identity components so punctuation cannot alias a structural separator. Every containment and dataset-relationship edge has an explicit stable ID; therefore two differently named semantic relationships between the same dataset pair survive as distinct edges instead of being collapsed by the ordinary builder's structural deduplication policy. That statement describes the constructed `Graph`. Persistence keeps both only on backends that support explicit edge IDs; structurally keyed stores collapse edges with the same source, label, and destination.

The output does not introduce a special semantic storage protocol. It is an ordinary `Graph` containing `SemanticModel`, `SemanticDataset`, `SemanticField`, and `SemanticMetric` nodes plus containment and `RELATES_DATASET` edges. It can be replay-compared, queried through the reference engine, or persisted through any backend that supports those ordinary graph operations, subject to that backend's documented multi-edge capability. Artifact-specific adapters remain responsible for parsing a source file and computing the hash supplied to the projection.

The conformance fixture is not synthetic: the test loads the packaged Apache Ossie TPC-DS YAML from upstream commit `ddb19f1b135a61c65603f4823a3526e2fab00cf1`, verifies

SHA-256 `bafbdc9d0e304ab22a40592f2b6bdfd45cc399c566533cd71343d33380c0d6e1`, parses the document, and replay-compares the resulting five datasets, 31 fields, five metrics, and four relationships. The published `grust-graph` archive includes Apache Ossie's `NOTICE` and Apache-2.0 text beside that exact fixture. `scripts/verify-package-attribution.sh` inspects the generated crate archive so release packaging fails if any of those three files is absent.

The key architectural point is that schema is metadata about a Grust graph, not a replacement for the graph. Application code can begin with plain graph construction, add schemas when operational needs demand it, and still speak the same store trait.

12 Design Tradeoffs

The universal node/edge layout appears in multiple backend plans because it is the easiest way to preserve arbitrary property graphs:

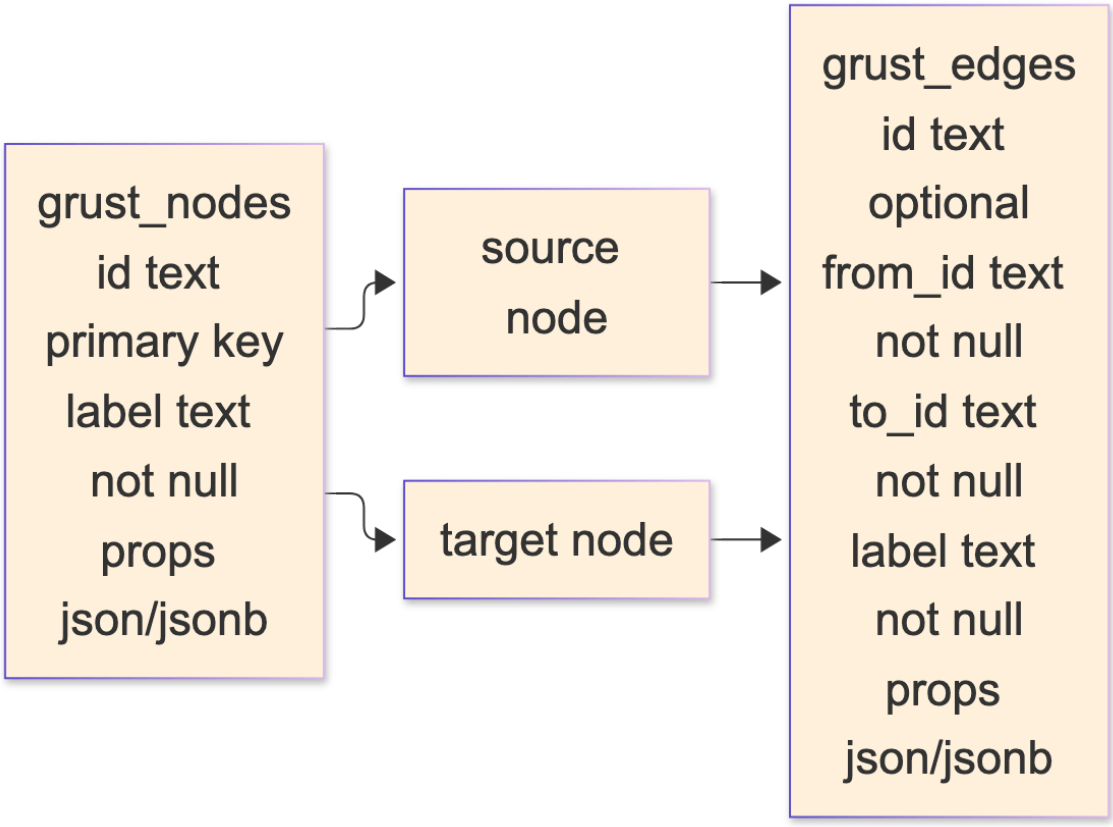


Figure 7: Diagram 7

The cost is that property typing and label-specific optimization require extra work. A label-partitioned backend can use stronger types and better indexes, but it needs schema, migrations, and more planning.

Grust's current architecture keeps both paths open. Core stays universal. Backends choose their storage layout. Schema support can become richer without forcing every backend to look the same.

13 Where Grust Can Grow

The next natural step is to deepen graph-native read and traversal support across the backends. HelixDB, LadybugDB, and SurrealDB now satisfy the portable `GraphStore` read and traversal surface, while FalkorDB remains primarily a write and indexing adapter. Further work can push more traversal work into backend-native query forms and add richer result shapes.

Traversal can also grow carefully. Property filters, bounded depth, path returns, shortest paths, and aggregation are all tempting. The important rule is to extend the IR only when several backends can implement the concept without smuggling database-specific query strings through the abstraction.

Incremental mutation uses an extension trait for backends that can apply graph deltas. Its operation model starts with element upsert and deletion and also contains the typed node/edge patch, matched update, property removal, and row-producing operations used by the portable Cypher planner:

```
pub enum GraphMutation {
    UpsertNode(Node),
    DeleteNode(NodeId),
    UpsertEdge(Edge),
    DeleteEdge {
        from: NodeId,
        label: Label,
        to: NodeId,
    },
    // Patch and matched-operation variants omitted here.
}
```

That serves CocoIndex-style target-state systems, streaming pipelines, and ordinary applications that need to apply deltas instead of replacing whole graphs. The default `apply_mutations` implementation is ordered but not atomic: if a backend uses the default and a later mutation fails, earlier mutations may already be committed. Backends with real transaction support can override that method. The PostgreSQL and pgGraph stores wrap mutation batches in PostgreSQL transactions, Turso wraps them in a local SQL transaction (which `querygraph-memory` relies on for atomic supersede-and-replace consolidation), and the SurrealDB HTTP and SDK stores wrap mutation batches in SurrealDB transactions. PostgreSQL and Turso's non-returning Cypher plan executors also reject unsupported lowering before writing, then execute the supported operations in source order inside one isolated transaction. The generic write-with-`RETURN` helper remains sequential because later operations may use intermediate bindings; it is not a whole-statement atomicity boundary. Explicit transaction scripts batch supported mutations when atomicity is required.

14 Compact snapshots and multigraph persistence

Acorn also changes how the Memory backend stores and exposes graph data. Node IDs and labels are interned as 32-bit handles. Each edge has one compact record, with 32-bit adjacency slots; optional edge IDs and properties live separately. Public reads retain their established node-ID and edge-key order. These are storage choices behind the same property-graph API.

`MemoryGraphStore::indexed_snapshot()` shares immutable store storage rather than copying every node and edge into a second graph. The index owns its slot ordering and adjacency. A write copies the store only while an external snapshot still holds the previous storage; older snapshots remain unchanged.

`GraphSnapshotSource` provides immutable slot access for `TypedGraphIndex`. `TypedGraphIndex::from_source` accepts a source whose answers remain stable for its lifetime; `TypedGraphIndex::new(Arc<Graph>)` remains available. Node and edge accessors may construct one element on demand. Calling `graph()` explicitly materializes and caches a full Graph when the source does not already own one.

Ordinary indexed Cypher reads traverse that source in place, preserving candidate order, result/error behavior and budget checks. Registered local-snapshot procedures still require a Graph; CALL therefore materializes the source only when the resolved provider declares `GraphRequirement::LocalSnapshot`. Graph-free providers such as `tvf.range` and `db.procedures` receive no snapshot and cause no full-graph copy, in both streaming and materializing query pipelines. Lean indexed MATCH execution does not imply zero-copy algorithm projection. Callers continue to own admission of retained input snapshots and this explicit materialization boundary.

Turso now preserves parallel edges by endpoint, label and optional edge identity. Re-putting one identity updates that edge; an absent ID and an empty ID are distinct. Bootstrap migrates the previous edge-key layout transactionally, retaining stored IDs so subsequent updates address the same migrated edges. PostgreSQL keeps its existing schema through the SQL dialect hook's default behavior.

Turso MVCC bulk loads commit groups of 20 SQL batches. Earlier committed groups remain if a later group fails: MVCC `put_graph` is no longer an all-or-nothing load. WAL mode retains its whole-load transaction. This changes the bulk-load boundary, not the transaction contract of `apply_mutations` or explicit mutation scripts.

LanceDB can retain a resident snapshot of its node and edge tables. The snapshot is built on a second read at unchanged table versions and serves anchored reads and traversal through interned IDs and adjacency slots. A version change sends reads back to the tables. Cache generations also prevent a build started before table recreation from repopulating the cache with old rows. `with_read_snapshot(false)` disables the mirror for all clones of that store. This trades retained process memory for fewer scans.

Turso bulk loads bind prepared multi-row upserts instead of retaining SQL text for the entire graph. Missing, empty and explicit edge identities use the same encoding as incremental upserts.

Ordinary connections allow a lazily filled 1 GiB page cache; synced connections retain their existing configuration. MVCC commits node groups before edge groups, with up to 20 batches per transaction. Loads over several MVCC writers, the fill through WAL, group commit for concurrent writes, and what each measured are the subject of the chapter "Turso under strain".

The Linux footprint example lives with its Cypher consumer: `cargo run --release -p grust-cypher --example memory_footprint -- 10000000`. This keeps the Memory crate independent of Cypher, including its packaged examples and development dependencies.

15 Turso under strain

This chapter documents the Turso adapter as it stands on the `turso-mvcc-concurrency` branch (pull request #6, head `abea518`), which is the adapter the next release ships. Everything here was measured; where a number depends on the engine version, the chapter says which version. The pinned engine is Turso 0.7.2 from crates.io. Upstream `main` (0.8.0-pre.11 at the time of measurement) is discussed separately, because it changes two of the conclusions.

15.1 Two journal modes, two contracts

`TursoConfig::journal_mode` selects the concurrency model, and the choice decides what the store does when several writers touch the same node.

`TursoJournalMode::Wal` is Turso's write-ahead log: one writer at a time. It is the faster loader by a wide margin, and under contention it does the honest single-writer thing: it accepts one writer's transaction and refuses the rest with a typed busy error. A refusal is not a lost write, so a WAL store passes the harness's hot-node scenario, but it does so by accepting between 20 and 120 of 3,200 attempted writes.

`TursoJournalMode::Mvcc` enables multi-version concurrency control (`PRAGMA journal_mode = mvcc`). Writes run inside `BEGIN CONCURRENT` transactions with bounded conflict retry, so concurrent writers make progress. On every graph measured, an MVCC store accepted all 3,200 hot-node writes. It loads more slowly, and it holds more memory while loading, because every row version of an open transaction stays resident until it commits.

Turso 0.7.2 converts between the modes on a live database. A database loaded in WAL mode and reopened as MVCC is accepted; a live MVCC store switches to WAL and back with `PRAGMA journal_mode`. `tests/wal_to_mvcc.rs` and `tests/wal_bulk_load.rs` pin both directions, and both pass against upstream `main` as well. The earlier documentation that said an existing WAL database is not converted was wrong for 0.7.2 and has been removed.

Choose by workload. A store that is loaded once and then read is a WAL store. A store that takes concurrent writes after loading is an MVCC store, and the rest of this chapter is about making its load cost bearable.

15.2 Loading an MVCC store

`put_graph` on an MVCC store no longer runs one transaction around the whole graph. That design kept every row version of the load in memory until the end and retried all of it on a conflict; the strain harness measured about 1,500 edges per second that way. Loads now commit in groups of

`MVCC_LOAD_COMMIT_STATEMENTS` batches, so a failure leaves the groups committed before it, and an MVCC `put_graph` is no longer all-or-nothing. WAL keeps its whole-load transaction.

15.2.1 Parallel writers

`set_mvcc_load_parallelism(writers)` runs the load over `writers` connections, each opened through `connect_shared()`, each taking a disjoint slice of the nodes and then of the edges under its own `BEGIN CONCURRENT` transactions. Turso executes one connection on one thread, so this is how a load uses more than one core; it needs a multi-threaded Tokio runtime. The harness default is four writers.

Free-running writers defeat Turso's automatic checkpoints: a checkpoint needs the checkpoint lock, every open transaction holds it in read mode, and with several writers there is always one open, so the checkpoint returns busy and row versions accumulate. A web-Google load reached 14 to 15 GB that way. The writers therefore advance in rounds of `MVCC_PARALLEL_ROUND_GROUPS` commit groups; between rounds, with nothing in flight, one `TRUNCATE` checkpoint folds the round into the database file. `set_mvcc_load_round_groups` adjusts the round size; smaller rounds trade time for memory, and the sweep found no speed to be gained from larger ones.

The effect, on one host with four writers, against the single-writer loader:

graph	edges	one writer	four writers
web-Google	5.1 M	16,013 edges/s	30,985
cit-Patents	16.5 M	5,526	26,661
soc-Pokec	30.6 M	4,698	26,385
GAP-road	57.7 M	2,928	27,708
sx-stackoverflow	63.5 M	2,929	13,464

Peak memory during a load fell from about 15 GB to 3 to 7 GB at web-Google scale. Scaling from one to eight writers measured 326, 220, 171 and 142 seconds for web-Google on an 8-vCPU host: about 35% of the load is serialized, in the engine's commit path.

15.2.2 The fastest fill: through WAL

`set_bulk_load_via_wal(true)` makes an MVCC store's `put_graph` switch the database to WAL, load everything in one transaction, checkpoint, and switch back to MVCC, whether or not the load succeeds. Journal mode is database-wide, so nothing else may write during the load; a bulk load before serving is the intended use. Handles opened later with `connect_shared` inherit the setting.

This is exactly WAL speed, and WAL loads are 1.5 to 1.7 times faster than four MVCC writers on Turso 0.7.2. On upstream `main` the gap is wider (see below), and the via-WAL fill is 2.65 to 3.1 times the parallel MVCC load. The round trip costs nothing measurable over a plain WAL load at five to seventeen million edges.

15.2.3 Foreign keys are off during a load

The Turso schema declares edge endpoints as `REFERENCES nodes(id) ON DELETE CASCADE`, and `bootstrap` turns `PRAGMA foreign_keys` on for the bootstrap connection. Until commit `3f1ed19`, that made the result of a load depend on the writer count: WAL and single-writer MVCC loads ran on the bootstrap connection and rejected an edge whose endpoint node was not stored, while parallel MVCC writers, which never set the pragma, accepted it.

`put_graph` now turns foreign keys off on the bootstrap connection for the duration of the load, outside any transaction, and restores the previous setting afterwards, whether or not the load succeeded. Every path agrees, and the behavior matches the Memory reference, where a vertex can exist only as an edge endpoint. `ON DELETE CASCADE` fires on the connection that deletes, which keeps the pragma on, so serving semantics do not change: a single `put_edge` to a missing node is still refused. `tests/dangling_edges.rs` loads a dangling edge through WAL, one MVCC writer and four MVCC writers, reads it back identically on each, and checks the restored strictness.

The two endpoint probes per edge were costing more than a third of WAL's load throughput. With foreign keys off, WAL loads of web-Google rose from 42.9 k to 70.3 k edges per second on Turso 0.7.2 (about 60%); the parallel MVCC path, whose writers were already unchecked, did not move.

15.3 Concurrent writes and group commit

Under the default `synchronous = FULL`, every MVCC commit fsyncs the logical log while holding Turso's global commit lock. Sixteen writers each attaching 200 edges to one hot node took about 22 seconds on 0.7.2; Neo4j does the same work in three to four.

`with_group_commit()` batches concurrent single-statement writes into one transaction: one fsync per batch of up to `GROUP_COMMIT_MAX_STATEMENTS` statements, and every acknowledged write is durable when it is acknowledged. It is not a relaxation of `synchronous`. Handles from `connect_shared` share the store's committer. On 0.7.2 it takes that workload from 21.1 to 3.2 seconds on one host, with all 3,200 writes accepted and no conflicts.

`set_synchronous` exposes the fsync policy per connection. Bulk loads already run at `NORMAL` internally (they commit rarely and end in a `TRUNCATE` checkpoint that fsyncs, so the load is durable when `put_graph` returns), so the setting matters for single-statement writes, where `NORMAL` removes the per-commit fsync at the cost of durability on power loss.

15.3.1 Version scoping: this changes on Turso 0.8

Upstream `main` has engine-level group commit, on by default (`PRAGMA mvcc_group_commit`, store-wide). There, Grust's client-side committer becomes a second coordination layer and costs about 19%: the same 16-by-200 workload takes 2.62 seconds with the engine's group commit alone and 3.2 seconds with `with_group_commit` on top. The client committer's numbers are identical on both engine versions, which is the point: it sets the ceiling.

`set_mvcc_group_commit(enabled)` exposes the pragma. It errors on 0.7.2, which has no such pragma. The intended policy is: on 0.7.2, use `with_group_commit`; on 0.8, leave the engine's group commit on and do not wrap. The default of `with_group_commit` should flip when the pinned

engine moves. The pin stays at 0.7.2 until 0.8.0 is a stable release, because the facade must not force a pre-release on downstream crates.

15.4 Allocator

The `turso` crate enables `mimalloc` in its default features, on 0.7.2 and on `main`: it installs mimalloc as the `#[global_allocator]` of the whole process. `grust-turso` depends on `turso` with default features off, because a library should not choose the process allocator, and exposes the feature as `mimalloc = ["turso/mimalloc"]`; the facade forwards it as `turso-mimalloc`. So a Grust application runs Turso on the platform allocator unless it opts in, while a plain Turso build does not. Measured on four-writer MVCC loads of web-Google, same host, alternating pairs: +14% on `main` and +16% on 0.7.2; on WAL loads +9% and +16%. Every number in this chapter was taken on the platform allocator; the strain harness has since made mimalloc its process-wide default and tags such rows `alloc=mimalloc`. A deployment that owns its binary should turn it on.

15.5 What was measured

The adversarial-graph strain harness loads real SNAP graphs and runs hot-node fan-out (A1), deep paths (A2), sixteen concurrent writers against one hub (A4), guarded-commit replay (A7) and an operability probe (A12), under nine hard gates. Every cell in this section passed with every gate at zero.

15.5.1 Turso against Turso, same host

On quegee (a c5n.4xlarge with no burst credits and less than half a second of CPU steal per run), the three largest graphs, same binary:

graph	MVCC load	WAL load	MVCC A4		WAL A4
com-Orkut, 117.2 M edges	16,046 edges/s	24,666	3,200/3,200 in 1.82 s		61/3,200
soc-LiveJournal1, 69.0 M	19,749	31,256	3,200/3,200 in 4.81 s		119/3,200
sx-stackoverflow, 63.5 M	18,504	31,243	3,200/3,200 in 6.54 s		50/3,200

WAL loads 1.5 to 1.7 times faster and then refuses 96 to 98% of concurrent hot-node writes. MVCC with group commit accepts every one. These WAL rows enforced foreign keys and the parallel MVCC rows did not, so the load ratio understates WAL; with both sides unchecked, WAL's lead is larger. The MVCC rows in this table and the next ran eight writers at `synchronous = NORMAL`; their A4 times measure conflict retries rather than the commit path and are shown for the acceptance count only. The durable four-writer lane did com-Orkut's A4 in 5.29 s, all 3,200 accepted, on a different host.

The durable seven-graph MVCC ladder on grust (the harness default: `synchronous = FULL`, group commit, four writers) is clean on all four core families on every whole graph up to com-Orkut.

It was the first MVCC run to reach com-Orkut at all: the single-writer loader would have taken most of a day.

15.5.2 Against Neo4j, same host

Neo4j 5.26 Community over Bolt in a 6 GiB container, quegee, zero steal:

	Neo4j	Turso MVCC	Turso WAL
com-Orkut load	16,800 edges/s	16,046	24,666
com-Orkut A1 / A2	144.5 s / 4,082 s	138.1 s / 1,312 s	96.8 s / 943 s
com-Orkut A4	2.90 s, 3,200/3,200	1.82 s, 3,200/3,200	1.05 s, 61/3,200
com-Orkut peak RSS	4.9 GB	22.5 GB	12.4 GB
soc-LiveJournal1 load	28,062	19,749	31,256
soc-LiveJournal1 A1 / A2	41.8 s / 2,501 s	29.5 s / 1,065 s	19.5 s / 773 s
soc-LiveJournal1 A4	2.90 s, 3,200/3,200	4.81 s, 3,200/3,200	1.29 s, 119/3,200

Turso wins traversal on every graph. A7, guarded-commit replay, is not compared: it runs only on stores with a Grust `GraphCommitStore`, and the harness drives Neo4j through its own Cypher without one, which is a gap in the harness path, not in Neo4j. Load splits by graph: on com-Orkut the MVCC store matches Neo4j's load; on soc-LiveJournal1 Neo4j loads 1.4 times faster. The A4 column is not a durability-matched comparison, because Neo4j commits durably and these MVCC rows ran at `synchronous = NORMAL`; in the durable same-host pairs the strain page counts, Neo4j wins every hot-node-write pair against the MVCC store and loses five of six against WAL, which accepts far fewer writes. Neo4j is three to five times leaner in memory throughout. Neo4j has no rows for the two smallest graphs, and the cit-Patents deep-path cell is vacuous on every backend (no path reaches the depth), so it is never cited.

15.5.3 Upstream `main`

Grust compiles and passes all 80 `grust-turso` tests unchanged against Turso `main` at `19710d58d` (0.8.0-pre.11, 1,859 commits past 0.7.2). Measured against 0.7.2 on the same host with alternating pairs and the engine version asserted in every probe:

- WAL loads are 22% faster on `main`; the B-tree write path improved.
- Concurrent single-statement writes are better on `main`, as described above.
- MVCC bulk loads are slower on `main`, and the gap grows with the graph: 5% at one writer and 15% at four on web-Google, 26% at four on cit-Patents, and 20 to 23% at four writers on sx-stackoverflow, soc-LiveJournal1 and com-Orkut on the same host. Reads on those stores are 4 to 24% faster on `main`; only the write path regressed.
- The cause is not group commit, not checkpointing and not the allocator; each was ruled out by A/B. A profile puts it in the value comparison inside the MVCC index-key compare (`types::cmp_in_column` and `read_value_serial_type`, about 31% of the load on `main` against

25.5% on 0.7.2). Upstream pull request #8385 rewrites exactly that path: against its own base it is 29 to 30% faster on this load, and under it the index-key share collapses to a single 12% symbol.

The recommended configuration on `main`, all of it existing switches: fill MVCC stores through WAL, serve with the engine's group commit and without `with_group_commit`, build with `turso-mimalloc`, and load with foreign keys off, which the branch already does.

15.6 Methodology, because it decided several numbers

Benchmark hosts that are burstable instances (`t2` on AWS) throttle silently once their CPU credits are spent; the throttling shows up as CPU steal, not as an error. One seven-graph lane ran a second engine "1.7 times slower" for that reason alone and was discarded. Every number in this chapter was taken with steal recorded, and every engine or configuration comparison on a burstable host was run as alternating pairs within one run so both sides saw the same throttling. Absolute throughput comes from the non-burstable host only.

Hot-node write times at `synchronous = NORMAL` vary from 1.4 to 5.2 seconds on both engines with no consistent direction; they measure conflict-retry luck, not the commit path, and are not cited. The write-contention comparison is the `FULL`-sync alternating A/B.

`/tmp` is a RAM filesystem on the benchmark hosts; every load timing here is on disk. Run-to-run drift on the same host is about 2%, so any effect under about 4% needs alternating pairs, and several plausible ideas were rejected on that basis: deferring secondary indexes during a load (slower), a plain `INSERT` for loads into empty tables (4% slower), a smaller writer page cache with GC off (flat), larger rounds (memory, not time), and dropping the edge identity column from the key (a real 2 to 4%, not worth a stateful dialect).

15.7 Limitations

The MVCC store holds more memory than the graph: 13 to 23 GB at 60 to 120 million edges with four writers, against 5 to 7 GB for Neo4j. The via-WAL fill needs exclusive use of the database for the duration of the load. A parallel MVCC load is not all-or-nothing. Foreign keys are not enforced during loads, by design. `set_mvcc_group_commit` is meaningful only on Turso 0.8. And the engine's global commit lock, not Grust, is what caps parallel MVCC loads at roughly 80% of WAL speed on 0.7.2.

16 Graph analytics through one Rust implementation

Acorn adds graph analytics to Grust's backend-neutral property-graph API. Applications prepare an explicit immutable snapshot, run a Rust kernel, and choose Rust, Arrow or ordinary Cypher consumption. Algorithms do not own a database connection or choose another graph. Memory and private Turso snapshots demonstrate the same prepared query against two independently captured sources.

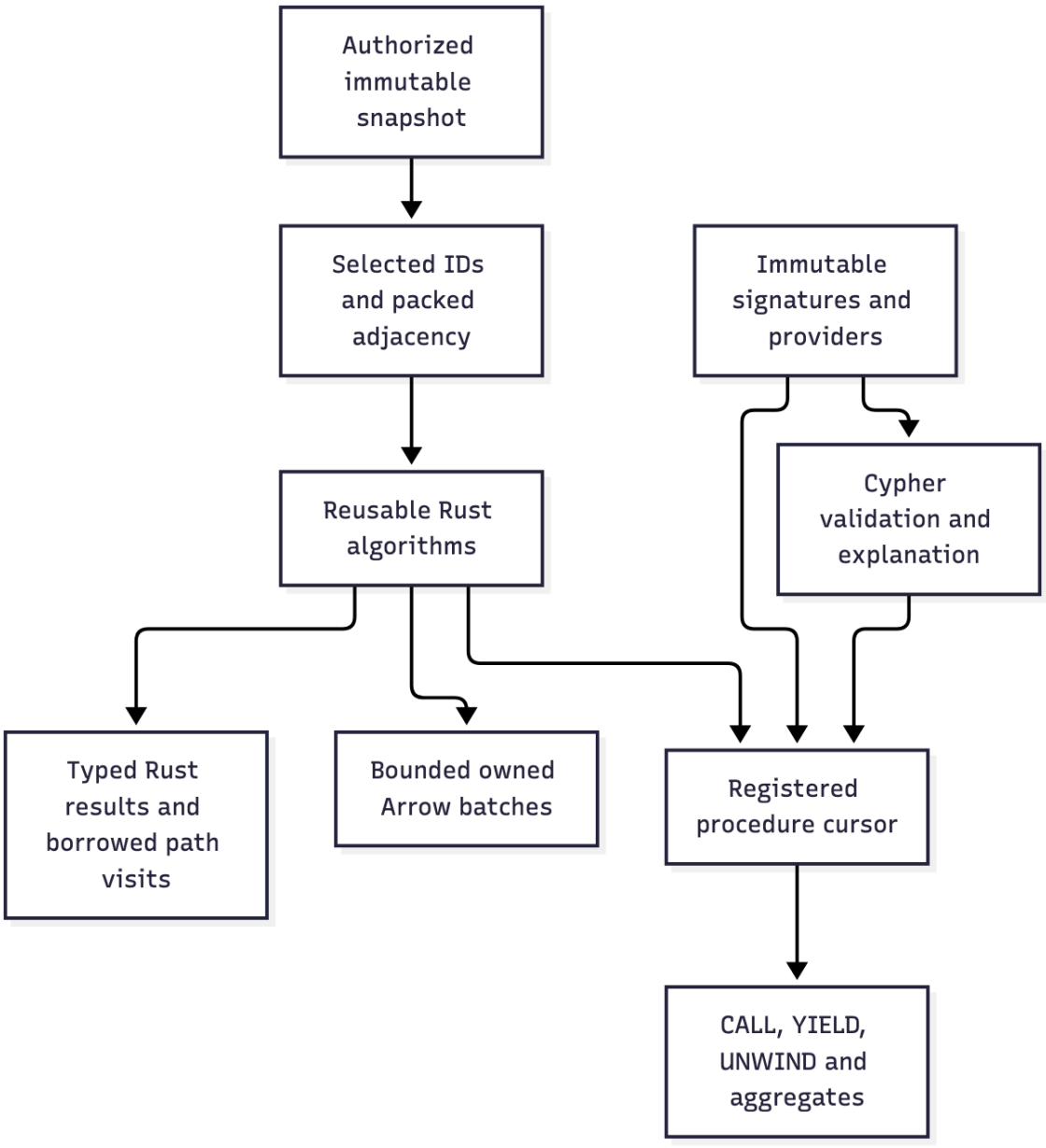


Figure 8: Diagram 8

`grust-algorithms` has no Cypher dependency. `grust-procedures` owns signatures, providers, checked cursors and the shared execution budget. The `grust-algorithm-procedures` crate connects them. An application can register an independent provider without changing the parser, analyzer or executor. The facade exposes these through `algorithms`, `cypher` and optional `arrow` features.

16.1 Starting from the facade

```
[dependencies]
grust = { package = "grust-graph", version = "0.24.1", features = ["algorithms",
"cypher", "arrow"] }
```

Register the desired providers once and retain the immutable registry:

```
use grust::{algorithm_procedures, procedures, CypherParameters};

let mut builder = procedures::RegistryBuilder::default();
procedures::register_builtins(&mut builder)?;
algorithm_procedures::register_algorithms(&mut builder)?;
let registry = builder.build();
let result = grust::run_read_query_with_registry(
    &graph,
    "roads",
    "CALL grust.algorithms.dijkstra('start', {weightProperty: 'cost'}) \
     YIELD nodeId, distance RETURN nodeId, distance",
    &CypherParameters::new(),
    &registry,
)?;
```

Here `graph` is a caller-owned `grust::Graph`. A bounded query additionally opts into `ReadQueryPolicy::allow_read_procedures`; catalog permission alone cannot admit analytics. For a backend capture, construct a `LocalSnapshot` after access checks, with an explicit graph, revision and principal. A prepared plan rejects a different graph name before invoking a provider. The capability pins data; the identity strings do not grant access.

For direct Rust, create an `ExecutionContext` with memory/work limits, batch size and optional deadline; `ExecutionContext::with_accounting` runs with less checking, as described under "Running without accounting". `GraphProjection::from_graph` accepts label selection, orientation and a weight policy. `from_topology` accepts external IDs and typed edges; `from_arrow_batches` reads typed columns directly, with node ids and edge endpoints either all Utf8 or all Int64. Integer endpoints are resolved through a direct table when the ids are compact, or a sorted lookup when they are not, instead of a string hash an endpoint, and in parallel when the execution asked for workers; a node's external id is then its id's decimal text, so results and kernel sources read as before. Kernel results share the projection, so dropping the caller's projection handle cannot invalidate IDs.

Beside the adjacency a projection keeps its edge table, which kernels index by edge *slot* to recover an edge's endpoints, its original ordinal and its external id. The table is columns: two four-byte endpoints an edge, an ordinal column only when some edge's ordinal is not its slot, and an id column only when some edge has an id. A graph handed over whole, with no label selection and no edge ids, therefore keeps eight bytes an edge. `GraphProjection::edges()` returns a view over it — `len`, `get(slot)`, `ordinal(slot)`, `iter()` — that hands out `EdgeRef { source, target, ordinal, id }` by value. The map from external id to node row, which only a kernel that takes a source by id reads, is built by the first such kernel; its work and its bytes are charged when the

projection is built, so a budget decides as it did. Runnable source examples are `weighted_paths` in `grust-algorithms` and `custom_procedure` in `grust-cypher`.

16.2 Available operations

All procedure names below have the prefix `grust.algorithms.`. They call the same kernels as direct Rust and have typed Arrow result adapters.

Operation	Contract
`bfs`	Hop distances from one external source ID
`multiSourceBfs`	Minimum hop distance from a nonempty source array
`dfs`	Deterministic reachable-node discovery order
`dijkstra`	Distances with finite nonnegative weights
`shortestPaths`	One selected full shortest path per reachable target
`wcc`	Weak components, including isolates
`scc`	Directed strong components with iterative traversal
`degree`	Exact projected arc counts and optional weighted strength
`pagerank`	Weighted scores, residual, iterations and convergence status
`articleRank`	As `pagerank`, with the mean outgoing weight added to every divisor, so a sparse citer confers less
`topologicalSort`	Complete DAG order or a concrete closed cycle witness
`bellmanFord`	Distances where weights may be negative, or the negative cycle that makes them meaningless
`astar`	One shortest path to a named target, guided by great-circle distance from two coordinate properties
`yens`	The `k` shortest loopless paths to a named target, ranked by `pathIndex`, with ties broken by node order
`allPairsShortestPaths`	Distance for every reachable ordered pair, streamed one source at a time and never held as a matrix; unreachable pairs are omitted, and `sourceNodes` restricts the sources
`longestPath`	Heaviest path ending at each node of a DAG, or the cycle that leaves it undefined

16.2.1 Centrality

Operation	Contract
`betweenness`	Ordered-pair dependency per node, exact or from a seeded sample
`closeness`	Reach over mean distance, per component, optionally Wasserman-Faust corrected
`harmonic`	Sum of reciprocal distances, needing no convention for disconnected graphs
`eigenvector`	Principal eigenvector by shifted power iteration, with convergence evidence
`katz`	Attenuated walk count, with convergence evidence
`hits`	Hub and authority scores, each at unit length

16.2.2 Community and structure

Operation	Contract
`louvain`	Modularity communities, named by their smallest member
`leiden`	As Louvain, with every community guaranteed connected
`labelPropagation`	Communities by weighted majority, on a reproducible schedule
`k1Coloring`	A colour per node so that no edge joins two of the same, greedily
`modularity`	Modularity and conductance of a partition the caller supplies
`kCore`	Core number per node and the graph's degeneracy
`triangleCount`	Triangles per node on the simple graph, and the total
`localClusteringCoefficient`	Triangles over possible triangles, null where undefined
`nodeSimilarity`	Node pairs by Jaccard, overlap or cosine over neighbour sets
`linkPrediction`	A score per node pair — common neighbours, Adamic–Adar, resource allocation, preferential attachment, total neighbours or same community — over every pair at distance two or a supplied list
`bridges`	Edges whose removal disconnects their endpoints
`articulationPoints`	Nodes whose removal disconnects two neighbours
`biconnectedComponents`	Edges grouped by the cycles they share
`spanningTree`	Minimum or maximum spanning forest, ties by edge ordinal
`maxFlow`	Maximum flow per edge from one source to one target
`minCut`	Which side of the minimum cut each node falls on
`fastRP`	A fixed-length embedding per node, from sparse random projection

`projectionStats` inspects selected topology. `estimateCsr` reports nominal adjacency-buffer upper bounds from snapshot counts, excluding graph storage, ID maps, original edge tables, kernel scratch, output and allocator overhead. Selection can reduce those counts. The estimate is not total memory admission.

The adjacency it sizes is narrow. A row bound, an arc target and an arc's original-edge slot are each four bytes, widened on read, so an outgoing arc costs eight bytes unweighted — a four-byte target beside a four-byte slot — and sixteen weighted; a reverse arc, which carries no edge slots, costs four unweighted and twelve weighted. Each CSR's row-bound array is four bytes per node plus one. A projection therefore holds at most `u32::MAX` nodes and at most `u32::MAX` arcs, and one that would pass either ceiling is refused with a named `AlgorithmError::Unsupported` at the checked add that would otherwise wrap, rather than falling back to eight-byte fields. Louvain's and Leiden's per-level scratch CSR keeps eight-byte offsets: it is a separate structure, not on a pull's per-node path.

The common configuration keys are `orientation`, `nodeLabels`, `relationshipTypes`, `weightProperty` and `defaultWeight`. Unknown keys fail. Null label arrays select all labels; empty arrays select none. Edges crossing outside the selected node set are excluded, while selected isolates remain. Orientations are outgoing, incoming and undirected. Undirected loops contribute one traversal arc; other undirected edges contribute two. Parallel edges remain distinct. Topology algorithms ignore weights after projection validation.

Weights must be finite and nonnegative. Integer weights outside `0..=2^53` are rejected rather than rounded. Missing or null weights fail unless a finite nonnegative default is explicitly supplied. Unit projections omit weight storage. Dijkstra uses strict improvement and stable adjacency order for equal costs; zero-weight ties cannot create predecessor cycles. Reachable cost overflow is an error. Unreachable distances are infinity in direct Rust and null in Cypher/Arrow. Component IDs are the external ID at the minimum projection row in each component.

`degree` returns every selected node, including isolates. Direct Rust `degree(&projection)` exposes exact `usize` counts and optional `f64` strengths. Arrow columns are `nodeId: Utf8`, `degree: Int64`, and nullable `strength: Float64`; Cypher returns an exact checked integer count and null strength for unweighted projections. No normalization is implicit.

```
CALL grust.algorithms.degree({orientation: 'incoming', weightProperty: 'cost'})
YIELD nodeId, degree, strength
RETURN nodeId, degree, strength
```

Counts include parallel and zero-weight arcs. Undirected loops count once under this projection contract. Weighted strength sums finite nonnegative weights; overflow fails explicitly. Negative weights remain unsupported. Unweighted execution reads existing CSR offsets in O(V); weighted execution takes O(V+A). Result buffers retain the shared memory admission, and bounded work-accounting chunks preserve cancellation polling without a lock per scalar. This contract does not establish universal degree-centrality compatibility with other engines.

PageRank defaults to damping .85, L1 tolerance 1e-8 and 1000 iterations. Scores start uniformly. Optional personalization controls teleportation and dangling mass; zero outgoing weight is

dangling. Parallel edges contribute separately. Finite large weights are scaled before sums. Iteration-limit results explicitly report non-convergence. The options are `damping`, `tolerance`, `maxIterations`, `personalization` and `precision`; `articleRank` takes the same. `precision` is `'f64'` by default or `'f32'`, and nothing else: at `'f32'` every score, per-arc probability, dangling mass, teleport share and base is formed and accumulated in `f32`, the residual is still summed in `f64` from the `f32` differences, and the Arrow `score` column is Float32 on every batch, with `iterations`, `converged` and `residual` unchanged. In Rust that is `pagerank_f32`, returning `PageRank<f32>`; `pagerank` returns `PageRank<f64>` as before. Both precisions are bit-identical at any worker count. The `f32` residual tracks the `f64` residual closely, so at 1e-8 the two usually stop at the same iteration; at tolerance zero `f32` reaches an exact fixed point sooner, and on a graph where most nodes are dangling it needs more iterations than `f64` at tight tolerances. A tolerance below one `f32` ulp of a moving score is met only at an exact fixed point: on a four-node graph with scores near 0.45 the default 1e-8 is below an ulp, the run oscillates by one ulp on two nodes for all 1000 iterations and reports `converged` false, while 1e-6 is met in 80. Seeds, community algorithms, negative-weight paths, all-pairs/k-shortest paths, flow, similarity and ML are deferred; the current catalog is not universal GDS compatibility.

16.3 Full paths and incremental consumption

A full path contains source-first external node IDs and cumulative costs beginning at zero. Original edge ordinals identify the selected multigraph edges. The zero-hop source path is included; unreachable targets are omitted.

```
CALL grust.algorithms.shortestPaths('start', {weightProperty: 'cost'})
YIELD nodeIds, costs
UNWIND range(0, size(costs) - 1) AS i
RETURN count(nodeIds[i]), sum(costs[i])
LIMIT 1
```

This query consumes every actual requested array element. LIMIT bounds the final aggregate output, not its input. The executor incrementally handles CALL/YIELD filters, simple WITH, UNWIND, plain RETURN and ungrouped COUNT/SUM/AVG. Generic array/index aggregation avoids allocating a binding row for every path element. It does not recognize a benchmark graph or substitute a closed-form answer. Other shapes use the existing budgeted materializing executor. Grouping, sorting, DISTINCT and collecting have no spill implementation and may exceed admission. Early LIMIT can stop consumption, but a global kernel may already have computed its result before producing the first batch.

One shared execution context accounts for projection buffers, scratch, provider results and downstream intermediates. Reservations stay attached through internal ownership transfers. Temporary consumed values release live admission; legacy materializing operators retain cumulative-copy accounting. Cancellation, deadline and work checks span preparation, kernels, reconstruction and consumption. Execution is synchronous without prefetch or parallel kernel workers.

Borrowed path visitor slices expire when the callback returns. Pull cursors reuse scratch storage. Owned scalar batches retain admission through wrapper clones. Brine 0.20.0's Arrow ownership

extension also retains algorithm-result admission through raw batches, buffer slices and nested children; shared physical buffers keep the original reservation without charging each clone again. Full paths and cycle/order arrays use LargeList offsets with explicit limits. An individual oversized path fails instead of truncating. Caller-owned input and externally retained legacy result tables require caller admission; logical accounting is not a process RSS sandbox for untrusted provider code.

16.4 Discovery, preparation reuse and execution classes

`db.procedures` describes the final registry generation, including argument and option defaults, outputs, provider identity, modes, correlation and computation boundary. Prepared explanations use the same definitions and consumer classifier. `explain_snapshot` also identifies the admitted revision and principal without running algorithms. These are Rust planning methods; a literal Cypher EXPLAIN prefix is not added by this extension.

Projection reuse is query-scoped and keyed by snapshot, principal, representation, selection, orientation and weight policy. Correlated CALL still executes once per incoming row; caching preparation does not cache algorithm answers. Reverse SCC adjacency is built lazily and stores only topology, without duplicate weights or edge identities. Projections retain their selection policy for inspection.

The current procedure executor accepts an explicitly selected local snapshot. Backend-native execution is rejected when unsupported, with no automatic whole- graph download. Memory and a private Turso database have snapshot integration proofs, including reads from old captures after later writes. Arbitrary external writers, remote snapshot stability and native analytics are not thereby qualified. Mutation/write-back needs a separate atomic transaction design and remains absent.

Both local direct Rust and ordinary Cypher completed the 65,536-node full-path chain, consuming 2,147,516,416 entries in each array. Those single observations have disclosed process envelopes and differ from the frozen Docker protocol. The repository retains independent small-graph oracles, source/binary receipts, frozen baseline results and current qualification status. See the coverage and resource contracts and benchmark evidence.

16.5 What the cooperative budget costs

Kernels charge the shared execution budget once per unit of graph work: per visited entry, and per step of a reconstructed path. That granularity is what makes a budget meaningful — an exhausted allowance stops the kernel where it stands rather than after the next batch — and it means the meter runs as often as the work it guards. On a 16384-node weighted chain with full path reconstruction, the meter is entered about 134 million times.

Two properties of that meter are therefore part of the algorithm contract.

Admission is lock-free and exact. The cumulative work counter and the cancellation flag are atomics. Each charge admits through a compare-exchange that recomputes admission against the value it actually replaces, so a concurrent charge cannot overshoot the limit between load and store, and an exhausted budget still fails exactly at its limit. Accounted memory and its high-

water mark are atomics admitted the same way, so a byte limit is as exact as a work limit. Only the cancellation wakers keep a mutex: they are rare, and they are the one place several fields must move together.

Memory had to follow work off the lock because of who charges it. A kernel reserves memory per batch, but a materializing Cypher consumer charges the logical bytes of every value it copies, once or more per row, through `charge_cumulative_memory` and `MemoryAccount::charge`. Those sample the deadline as work charges do; `reserve` still reads the clock. Two things follow from dropping the lock. `usage()` reads its figures one after another rather than as one snapshot, so read it after execution for exact totals; while an execution runs, `peak_bytes` is never below `live_bytes`. And a poisoned lock no longer fails a memory charge, because there is no lock to poison; only waker registration can still report it.

The deadline is sampled; everything else is not. An execution that sets no deadline pays nothing for deadline enforcement — neither a clock read nor a counter. An execution that sets one has its deadline observed within 1024 charges rather than on every charge, because reading the clock per unit of work costs more than the work itself wherever the host clocksource is paravirtualised rather than a register read. Cancellation remains an unconditional atomic load and is observed immediately. Budget limits remain exact. `checkpoint` reads the clock every time, so a caller that needs a precise poll has one.

The practical consequence for callers: choose a deadline deliberately. A bounded read policy requires a finite one, and a kernel that charges per entry will consult it often. If a caller needs expiry observed more tightly than 1024 units of work, `checkpoint` is the exact instrument; the sampled path is for the charges themselves.

These are contract properties rather than tuning knobs, and they are visible in measurements. In the companion algorithms benchmark, replacing per-charge mutex locking improved full-path Dijkstra by 14 to 29% on direct execution and PageRank by 23 to 29% across graph families, and sampling the deadline reduced a full-path Cypher query on a 4096-node chain from 21,721 ms to about 2,300 ms. Both figures come from one host with a Xen clocksource and are not portable constants; the boundary statements and raw evidence accompany them there.

16.6 Running without accounting

The budget's cost is real, and a library that does no accounting does not pay it. For a comparison with such a library to be like-for-like, an execution can be constructed with less checking, explicitly and by name:

```
use grust_algorithms::{Accounting, ExecutionContext, ExecutionLimits};

let context = ExecutionContext::with_accounting(
    ExecutionLimits {
        memory_bytes: 1 << 30,
        work_units: usize::MAX, // required: an uncounted budget cannot be
enforced
        batch_rows: 8192,
        deadline: None,
```

```
    },
    Accounting::WORK_UNCOUNTED,
)?;
```

`ExecutionContext::new` is unchanged and counts work. An unlimited budget is not an opt-out: `work_units: usize::MAX` still counts every unit and still reports the total. Two switches, independent because they give up different things:

Mode	Work counted	Cancellation and deadline	Memory admission
`Accounting::COUNTED` (default)	yes, budget exact	observed	enforced
`Accounting::WORK_UNCOUNTED`	no	observed	enforced
`Accounting { work: Counted, interruption: Disabled }`	yes, budget exact	not observed	enforced
`Accounting::UNCHECKED`	no	not observed	enforced

Uncounted work skips the shared counter, the block grants of work meters and the budget comparison, and a meter is never registered, so creating and dropping one takes no lock. What is given up is the work budget and the work total. Cancellation is still observed at every charge, and the deadline is still sampled at the counted cadence: once per 1024 charges on the context, once per block on a meter.

Disabled interruption additionally stops every charge, checkpoint and reservation from reading the cancellation flag or the clock. What is given up is the ability to stop a running kernel. It is a separate switch because it is the larger trade, and because the check is itself a measurable share of a charge-dense loop; neither is implied by the other.

Memory admission is never switched off. It is charged per allocation rather than per visited entry, so it is not the hot-loop cost, and it is what stops a projection from exhausting the host.

A limit the mode cannot enforce is refused when the context is constructed: a finite `work_units` with uncounted work, or a deadline with interruption disabled. On an execution with interruption disabled, `cancel()` returns `Unsupported` instead of accepting a signal nothing will observe, and a `cancelled()` waiter completes at once with the same error.

The mode is part of the result. `ExecutionContext::accounting()` and `ResourceUsage::accounting` name it, and `Accounting` displays as `counted`, `work-uncounted`, `uninterruptible` or `unchecked` for a report row. `ResourceUsage::work_units` is a `WorkCount`: `Counted(n)` or `NotCounted`, which is not zero; `ResourceUsage::counted_work()` returns it as an `Option`. Results are bit-identical in every mode, sequentially and at every worker count.

17 Graph algorithms under measurement

This chapter documents what the companion algorithms benchmark measured about this library, on current sources rather than a frozen snapshot. Everything here was measured on one host — an 8-vCPU Xeon E5-2686 v4 instance, two CPUs and 4 GiB per service, algorithm concurrency one — and where a number depends on that host, the chapter says so. Results from three source pins are kept apart and never pooled.

The benchmark's job is not to rank engines. It runs several execution classes over identical graphs and states what each timer contains, so a difference can be attributed to something specific. Twice in this round the attribution overturned the obvious explanation, which is the reason the chapter exists.

17.1 Execution classes, not engines

Six current columns run the same algorithms through different machinery.

Column	What its timer contains
direct	kernel and result conversion, projection timed separately
ordinary Cypher	parse, policy validation, projection, row consumption, with full-path distance verification timed apart
Arrow	row-to-Arrow conversion, native Arrow projection, Arrow result consumption
DataFusion	complete node and edge scan plans, then the same native kernels
Turso direct and Cypher	Grust kernels over a verified snapshot from a durable Turso database

The Turso columns are not Turso-native SQL graph algorithms, and their database loading is not an ingestion comparison against another engine's projection. The Arrow and DataFusion columns do not claim DataFusion executes graph kernels; DataFusion prepares input, and the kernels that follow are the same ones direct execution runs.

Because the classes do unequal work by construction, a ratio between two of them is a statement about machinery, not about speed in the abstract.

17.2 Full paths are an output contract

A shortest-distance query returns one number per destination. A full-path query returns the intermediate nodes and the accumulated cost along the route. On a chain, the path to the first node holds one entry, the next holds two, and across all destinations each array holds `n(n + 1) / 2` entries: 134,225,920 at 16,384 nodes and **2,147,516,416 at 65,536**. Every entry is constructed and consumed. Nothing is summed in closed form, no family is special-cased, and paths are reconstructed one at a time into reused buffers rather than retained together.

That contract is what makes the benchmark informative about this library, since it exercises reconstruction, the work meter and result conversion at a scale where each of them is visible.

17.3 The cooperative budget was the kernel's largest cost

Kernels charge the shared budget once per unit of graph work — per visited entry and per path step — which is what makes an exhausted allowance stop a kernel where it stands. On the 16,384-node chain the meter is entered about 134 million times, and profiling attributed **72.8% of kernel self time to `charge_work`**, against 14.8% for visiting paths and 6.6% for advancing path buffers. The meter took a mutex on every call, in a single-threaded kernel.

The counter and the cancellation flag are now atomics, with each charge admitted through a compare-exchange that recomputes admission against the value it replaces, so budgets are still enforced exactly and granularity is still per unit. Full-path Dijkstra improved by up to 29.4% on direct execution, except on the hub family, where it was flat at −2.9% and +1.1%; PageRank improved between 22.5% and 29.1% across every family.

A share of self time is not a share of removable wall time. The atomic still costs and the surrounding work is real, so a 72.8% profile share produced a 20 to 29% improvement, not a threefold one.

The charge later moved off the node as well. A reduction block's arc total is one subtraction on the transpose's row bounds, so PageRank's fused pull charges `block_nodes + block_arcs` once per 4,096-node block rather than `1 + in-arcs` per visited node. A completed kernel charges the identical total, so every budget decides exactly as it did; what moves is where a refusal lands, on a block boundary rather than a node boundary. The charge still precedes the work it names, so no work a budget refused is ever performed, and each block loop polls interruption explicitly on the cadence the per-node charges used to reach the meter's admission on — every 113 nodes on an eight-arc graph, every 1,024 on an arcless one — so cancellation and the deadline stay as responsive as they were.

17.4 Deadline enforcement is a policy, and policies differ

Ordinary Cypher's full-path query measured about ninety times direct execution. The query text was not the cause: removing the row expansion entirely, asking only for `count(*)` and `sum(size(nodeIds))` rather than one `UNWIND` row per entry, saved about 4%.

Profiling put roughly 83% of the remainder in the clock. A bounded read policy requires a finite deadline, the benchmark's Cypher participant disclosed a 24-hour ceiling, and the execution context therefore read the clock on every charge — about 8.4 million times at 4,096 nodes. Direct execution passes no deadline, never reads the clock, and completed the same work in 197 ms.

Two facts turn that into a policy difference rather than an engine difference. The host's clocksource is `xen` rather than `tsc`, so each read goes through a paravirtual clock instead of a register, which inflates any per-unit check on this machine specifically. And the compared engine samples its own termination check: its shipped artifact carries a 10,000-node check interval and a 10,000-millisecond flag interval, so it consults a cached flag between samples. This library checked every unit. The resulting difference was in how often each system asks, not in how fast either computes.

Charges now sample the deadline every 1024 units. An interval sweep showed the gain complete by 256 and flat thereafter, so the interval stays tight and remains an order of magnitude stricter than the 10,000-unit interval it is measured against. Cancellation is never sampled, budget limits still fail exactly at their limit, a memory reservation keeps an exact check while the per-copied-row memory charges sample as work charges do, and `checkpoint` reads the clock every time.

The first version of that change tracked its sampling counter even when no deadline existed, so kernels that set none — every direct, Arrow and DataFusion path — paid an atomic they had never paid before, and measured 13 to 46% slower until a paired run caught it. An execution without a deadline now returns before both the counter and the clock.

17.5 Correctness is an oracle, not an afterthought

Every sample in every sweep is validated against an independent C++ reference, and failures are retained rather than discarded. That is how a defect unrelated to performance surfaced.

A durable-loading experiment failed all 36 of its samples. Results were being associated with nodes by position in the snapshot rather than by node identifier. Bulk loading and single-writer loading insert in input order, so position and identifier coincided and the defect was invisible; four concurrent writers scrambled insertion order, and every distance landed on the wrong node. The values were an exact permutation of the reference — identical multisets, wrong positions.

Snapshot verification could not catch it, by construction: it compares sorted records because database scan order is legitimately arbitrary, which makes it blind to a permutation. The adapter now restores input order after verification, in a separately reported phase outside every algorithm timer. No published measurement was affected, because every published run used bulk loading.

17.6 Allocator and durable loading

The global allocator is a measured default rather than an assumption. A matched control differing only in six global allocator declarations found mimalloc faster in 41 of 48 cells, by as much as 27.8% on sparse random graphs, with three cells slower and reported, led by DataFusion on the chain at +5.4%.

Group commit is a property of durable loading, not of kernels, and is measured as its own preparation workload. Engine grouping roughly halves durable load time — 9,962 ms to 4,232 ms at 1,024 nodes — while the algorithm timers do not move at all. No group-commit setting may be credited as a kernel optimization. WAL declines the four-writer workload outright with a locked database, which is why the experiment is specified for MVCC only.

17.7 What the whole set moved

On the 65,536-node chain, across three pins, with the frozen historical binaries byte-identical throughout and moving by at most 1.4% — the control that makes the rest attributable to code rather than to the machine:

Column	baseline	with the meter and deadline work	with the executor work
direct	66,477	52,142	51,869
Arrow	241,622	184,957	184,291
ordinary Cypher	5,808,625	535,450	594,419

The full run fell from 3 hours 32 minutes to 35 minutes of wall time. Ordinary Cypher on that case moved from roughly fifteen times the compared engine's server query to about 1.55 times it, while remaining a different measurement: that engine's projection is built before its timer starts and this one's is inside it.

The reference-executor work that followed improved ordinary Cypher between 88.6% and 98.1% across every graph family at 4,096 nodes. The full-path chain was the one case it did not reach: it regressed there by 2.0% at 4,096, 9.8% at 16,384 and 11.0% at 65,536, because that case is dominated by materialising one heap-allocated string per path entry rather than by the per-row overhead the work removed.

Reporting that case led to the next change, which stopped deep-copying each yielded value into every row and began admitting work per path, or per 1024 steps, instead of per entry. Both the regression and the original cost went with it. On the 16,384-node chain, ordinary Cypher fell from 36,913 ms to 20,255 ms, direct execution from 3,190 ms to 1,298 ms, and Arrow from 11,612 ms to 9,397 ms, while the frozen C++ participant moved 0.4% in the same runs. At 4,096 nodes with five measured samples, direct execution improved 58.7% and ordinary Cypher 45.2%. The per-entry charge that the first profile found was therefore worth roughly a further 2.4x on direct execution once it was charged per path; the 65,536 case has not been re-run at that pin.

17.8 The same problem, one accounting axis over

List binding forms arrived in the same release as the reference-executor work. `reduce` parses and returns aggregates identical to the `UNWIND` form the benchmark uses, so the feature is correct, and it is slower by a constant factor rather than a scaling one: 4.6 to 4.9 times on the pin where it landed, and about 6.1 times on current code, where the row-expansion path improved faster than the fold did.

The cause is this chapter's own subject, moved one axis across. Work charges are lock-free now, but each folded element still evaluates through the general scoped evaluator, and every variable reference clones a value and charges its bytes through a memory account that takes the state mutex: two lock round-trips per element, plus a string allocation that is inherent to path node identifiers being a string type. Closing the gap needs byte accounting without the mutex — which is what work accounting already received — or a compiled fold.

That is worth stating as a contract property rather than a benchmark result. The budget has two meters, and only one of them has been made cheap. Until the other follows, a query that touches many small values per row pays for its own accounting, and the cheaper way to write it is the one that touches fewer.

Note also what the arithmetic did while nobody was looking. When the clock reads dominated, removing the row expansion was worth about 4% of the query, so the shape did not matter. On current code the same measurement is 1,533 ms against 718 ms without the expansion: the shape is now worth about half. Making the fold fast is no longer a rounding error, which is an argument for the binding-forms work that did not exist when it was proposed.

17.9 What the unchanged columns say

The frozen historical participants are this chapter's control: identical binaries across every run, moving at most 1.4%. The distance between two of them is nonetheless a result. The Rust participant finishes the 65,536 chain in 10,888 ms and the NetworKit-derived C++ one in 30,454, a factor of 2.8; at 4,096 the same pair is 1.55. A gap that widens with the work is not a constant-factor difference between compilers.

Profiling both attributes it to allocation rather than to computation. The C++ run spends 36.2% in path reconstruction, 15.2% in kernel page-fault handling and a further 10.7% between libc and kernel memory locking; the Rust run spends its time in the kernel routine and its consumer, with no allocator or kernel frame above 2%. The structural difference is that one stores predecessors as a vector of vectors and returns a freshly grown vector per reconstructed path, while the other walks a flat parent array into two buffers allocated once and reused.

The reason this belongs in a chapter about Grust's own contracts is that it is the same lesson as `charge_work` and the deep-copied yielded lists, arriving from outside. In each case the expensive thing was an interface that forced work per item — a lock per charge, a clone per row, an allocation per path — rather than the computation the interface was wrapping. A signature that returns a fresh container per item cannot reuse a caller's buffer, and no amount of care inside the implementation recovers that.

17.10 Limitations

These numbers are not portable. The deadline and clock findings are shaped by a paravirtual clocksource, and the same code on a host with a register clock would show a smaller penalty. The 16,384 and 65,536 results are single samples and establish completion, not stable ranking. Container memory peaks are whole-container figures including file cache and are never presented as per-participant resident memory. Results from different source pins are kept in separate sets. No result from the separate graph-query or strain benchmarks is combined with these.

18 Arrow pipelines, ADBC and DataFusion

Grust's Arrow-based adapters use `grust-arrow` for common batch and IPC operations. The graph core and other adapters remain independent of Arrow. LanceDB and Sail select Arrow 58; the private Ladybug adapter selects Arrow 55. The default interchange and ADBC helper use Arrow 59. These modules compile the same implementation against the native SDK types.

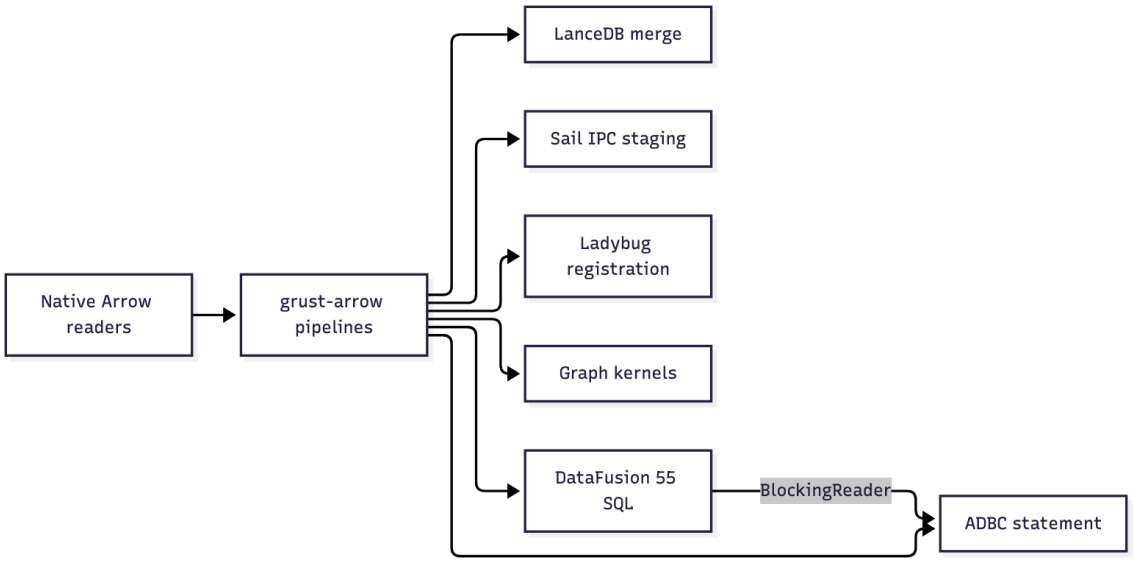


Figure 9: Diagram 9

18.1 Native tables and readers

`ArrowTable` retains arbitrary native Arrow columns, metadata and batch boundaries. Its schema is checked when constructed. Its `into_reader` method transfers ownership into the standard `RecordBatchReader` interface without concatenating batches or serializing their contents.

`BatchReader` wraps a standard reader with a row limit and an input-batch memory limit. It pulls on demand and slices without copying buffers. Errors terminate the stream after one error result. A slice can retain its complete original allocation. The memory limit applies to decoded Arrow arrays, not process RSS or the upstream decoder's temporary allocations. `ByteLimitWriter` separately bounds encoded output, including schema bytes, before the sink grows past its limit.

The shared IPC functions accept standard readers and caller-owned sinks. They support multiple batches and schema-only empty streams. IPC remains an explicit encoding boundary; it is not needed when producer and consumer already share native Arrow types.

18.2 Property-graph interchange

The original `ArrowGraph` is a validated pair of scalar-property batches:

Table	Structural columns
Nodes	non-null UTF-8 `node_id`, `label`
Edges	non-null UTF-8 `source`, `target`, `label`; nullable UTF-8 `edge_id`

Property columns use `property.<key>` and a non-null Boolean `present.<key>`. The marker distinguishes a missing property from an explicit null. Scalar properties support Null, Boolean,

Int64, Float64 and UTF-8, preserving integer precision. Mixed non-null types and complex graph properties are rejected by this scalar interchange contract rather than silently coerced.

```
use grust::arrow::ArrowGraph;
use std::fs::File;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let graph = grust::Graph::default();
    let tables = ArrowGraph::from_graph(&graph)?;
    tables.write_ipc(
        File::create("nodes.arrow")?,
        File::create("edges.arrow")?,
    )?;
    let restored = ArrowGraph::read_ipc(
        File::open("nodes.arrow")?,
        File::open("edges.arrow")?,
    )?.to_graph()?;
    assert_eq!(graph, restored);
    Ok(())
}
```

This legacy file-format API still stores one batch in each of two Arrow files. `ArrowGraphTables` supports the same graph contract across multiple batches. It validates global node identity and edge endpoints without building property rows or adjacency. It preserves isolates, loops, parallel edges and row order. `ArrowGraph::into_tables` and `ArrowGraphTables::into_tables` expose general Arrow tables whose readers compose with other consumers.

These graph restrictions do not constrain arbitrary `ArrowTable` values passed to ADBC or backend-native registration. Those pipelines retain nested, dictionary, temporal and extension columns according to the destination's capabilities.

18.3 Database ingestion

The shared universal storage layout uses `id,label,props` for nodes and `key,id,from_id,to_id,label,props` for edges. The property payload preserves Grust's tagged serialization, including complex values. Native storage encoders and decoders share the stable edge-key validation rule. Projection can rename columns without copying their arrays.

LanceDB's `load_arrow` takes standard readers in that storage layout and merges native batches, maintaining typed mirrors. Sail's `load_arrow` accepts its existing SQL column names and plain-JSON property representation. It reuses identity columns and performs required property normalization and endpoint-label resolution. Spark Connect transports the staged batches as IPC. Sail's graph IPC convenience loader now uses the same streaming path.

Stream graph loads validate a batch before writing it. Nodes precede edges, and earlier batches can remain committed if a later operation fails or is cancelled. They do not promise atomic whole-import replacement. Consult the adapter contract for existing-row upserts, constraints and typed mirror behavior.

Ladybug can register native readers without IPC, and visit query batches without collecting all output. Registration requires all input batches simultaneously; a conservative retained-memory bound applies before registering the table. Registration creates a queryable Arrow table, not an implicit persisted graph upsert. Persisted graph loading continues through the adapter's Arrow-backed COPY path and transaction policy.

18.4 ADBC and extension

Enable the facade's `adbc` feature, or `grust-arrow`'s optional `adbc` feature, to bind an Arrow 59 reader to a caller-owned ADBC statement with `grust_arrow::adbc::ingest`. ADBC controls connection lifecycle, target catalog/schema, ingestion mode, transactions and cancellation. The helper preserves driver errors and unknown affected-row counts. An ingestion mode such as append is not reinterpreted as graph upsert.

The optional `ffi` feature exports standard Arrow C streams through upstream Arrow's implementation and release callbacks. Grust adds no unsafe FFI code. Rust callers using the same Arrow major can share readers directly; IPC is an explicit option for serialized cross-language or cross-version boundaries.

New adapters should reuse the standard reader and shared batch operations, keeping only their schema and backend semantics locally. Enable the facade's `datafusion` feature for the shared DataFusion 55 foundation. It registers native Arrow 59 tables and validated graph catalogs without re-encoding their buffers, accepts upstream table providers, and exposes DataFrames and streaming SQL results. Working-memory, parallelism, batch size and spill policy are explicit. Input buffers and retained results remain caller-owned admission; the working pool is not a process RSS limit.

`BlockingReader` connects a result stream to synchronous Arrow and ADBC consumers on an ordinary thread or `spawn_blocking`, using a caller-owned live Tokio runtime. It adds no worker, prefetch queue or IPC boundary. Existing backend SDKs keep their own compatible Arrow and execution-engine versions. Cypher lowering and algorithm kernel selection require separate semantic and performance qualification; this foundation does not automatically change them.

See the repository's `docs/arrow-pipelines.md` for detailed admission, compatibility and testing contracts. Reproducible pipeline benchmarks measure native slicing, explicit IPC boundaries and graph validation separately from backend load and algorithm timings.

18.5 Typed Cypher execution

Combining facade features `cypher` and `datafusion` enables the bridge at `grust::datafusion::cypher`. Direct consumers can enable `grust-datafusion` feature `cypher`. It provides an explicit execution bridge over native Arrow graph tables. `GraphSnapshot::execute` accepts Cypher text, parameters and `OutputLimits`; it uses the existing parser and semantic analyzer, builds DataFusion 55 expressions directly, and returns an ordinary `CypherResultTable`. No SQL text or intermediate row-oriented graph is generated. Isopod 0.17.0 adds this explicit surface. Ordinary Cypher entrypoints do not yet select it automatically, and it does not implement the complete bounded read policy.

18.5.1 Snapshot and composition

`GraphSnapshot` captures validated immutable node/edge providers, independent of session catalog replacement. Existing Arrow buffers remain shared. An additional UInt64 ordinal, costing eight bytes per edge, supplies physical relationship identity within the snapshot. Parallel edges and optional/repeated external edge IDs remain distinct. Backend transaction and authorization identity, input storage and ordinal allocation remain caller responsibilities.

`GraphSnapshot::plan` selects the node or relationship compiler from the parsed query shape. `QueryPlan` records its `PlanKind` and either a DataFrame or an unsupported reason. `CypherExecution` distinguishes completed execution from an unsupported query that never executed. Parse, semantic, planning, execution and output-limit errors propagate without retrying another executor or snapshot.

Applications can also use `plan_node_scan`, `plan_relationship_scan`, and the snapshot's directed/undirected relationship operators directly. The shared `ExpressionBindings` contract resolves graph variables to typed physical expressions for scalar and aggregate compilation. Missing properties become null; unknown variables remain errors. This allows composition without copying the scalar semantics into each pattern planner.

18.5.2 Qualified language surface

The current compiler supports node scans and fixed-length relationship paths in either direction or undirected form; labels, relationship types and scalar inline maps; named, anonymous and repeated endpoint bindings; WHERE, projection, DISTINCT, count variants/grouping, integer/string MIN/MAX, node identity, projected ordering, and literal/parameter pagination. Scalar domains are Boolean, Int64, UTF-8 and null. Only Boolean true retains a WHERE row.

Undirected matching emits both orientations of non-loop edges and each self-loop once. Repeated endpoints constrain matching to self-loops using one node join; the undirected form omits its reverse branch. Anonymous names are generated after semantic analysis and cannot collide with explicit pattern bindings.

Variable-length paths, OPTIONAL MATCH, correlated maps, floating-point and mixed numeric expressions, arithmetic, additional aggregates and duplicate projection names still need mappings or result remapping. Integer SUM requires particular care: preserving an equal final total does not preserve sequential overflow behavior across partitions. Unsupported cases are not silently coerced.

18.5.3 Output and remaining policy work

`decode_result_batch` preserves column order/names, row multiplicity, nulls and exact Int64 values when converting native scalar batches to portable rows. Types are resolved once per column; unsupported types fail before allocating rows. Arrow/ADBC consumers can retain native batches and avoid owned row conversion.

`collect_result` checks cumulative row counts before decoding each batch and counts exact serialized JSON output bytes without allocating a JSON buffer. The count includes column metadata, delimiters, escaping and inter-row commas, including empty results. Exceeding either

limit returns an error with no partial result. One decoded batch still requires separate memory admission.

Mantis 0.19.0's `decode_result_batch_with_context` charges cumulative logical copy bytes before creating portable rows and strings. It measures the actual Arrow slice, including null-aware UTF-8 lengths, column names and row/value containers. `collect_result_with_context` also charges collection metadata and moved row containers, and shares cancellation/deadline control. Controlled `GraphSnapshot::execute_with_context` now uses this collector. The charge stays consumed after output is dropped; it is not a live-heap or allocator-capacity measurement. Arrow input, validation descriptors and operator allocations remain separate. No query is automatically routed on this basis.

DataFusion's working-memory pool and these output checks do not enforce Cypher's candidate-work, intermediate-copy, query/input or deadline contracts. Automatic selection remains pending until those boundaries and measured cost decisions are integrated. The completed scan profile reports conversion/preparation costs separately; the parallel-ring path profile also retains preparation and admission boundaries. Passing explicit execution tests does not establish backend-wide speed or resource-policy parity.

`RelationshipPlan::join_trail` composes plans through shared node bindings and excludes physical edge reuse across all joined parts. Parallel edges remain distinct. Plans must come from the same captured snapshot; cloned handles retain that identity. Physical columns are renamed before composition to avoid alias collisions. The parsed path planner composes these operators for fixed-length paths; variable-length lowering and path-resource admission remain separate work.

Fixed-length paths reuse the shared RETURN and predicate compilers after trail composition. Labels, relationship types, inline maps and repeated nodes constrain the complete path, with physical edge uniqueness across all segments. Direction may differ per segment. Variable-length bounds and named path values remain unsupported; the pinned parallel-ring profile measures two- and three-hop execution with separate preparation costs; it is not a general routing threshold.

18.5.4 Shared cancellation and deadlines (Ostracod 0.18.0)

`ExecutionContext::cancelled()` is a runtime-independent notification shared by procedures and relational execution. `grust_datafusion::run_cancellable` races an operation with that notification and the context's absolute deadline. `GraphSnapshot::execute_with_context` applies it through complete Cypher result consumption. The operation and its owned streams are dropped on cancellation or deadline; errors are never retried. Dropping one wrapper unregisters its waiter without cancelling sibling work. A deadline needs Tokio's time driver.

Control is cooperative: synchronous work within one poll is not preempted, and providers/kernels must checkpoint during computation. Wrapping stream creation alone does not control later consumption. These methods do not charge candidate work or intermediate allocations and do not establish complete read-policy admission or automatic routing.

`control_stream` retains cancellation and deadline control for an Arrow stream's whole lifetime. `DataFusionEngine::execute_stream_with_context` controls both SQL preparation and the re-

turned stream. Batches pass through without buffer copies, queues or worker tasks. Completion or the first error immediately drops the provider stream and timer; subsequent polls remain finished. The controlled stream can feed `BlockingReader` and ADBC ingestion directly.

18.5.5 Prepared read admission (Ostracod 0.18.0)

`grust_cypher::PreparedReadRequest` owns the validated AST, policy and original absolute deadline, borrows the admitted immutable parameters, and retains any application registry generation. The bounded reference executor uses its request, graph/index and output checks. Indexed graph checks reuse the cached exact serialized size; parameter/output counting does not allocate encoded JSON. Oversized parameters fail before graph inspection. Preparation does not authorize or bind a backend snapshot, install execution budgets, or qualify a DataFusion route. Route-specific candidate/intermediate accounting must still be enforced.

18.5.6 Exact snapshot statistics (Ostracod 0.18.0)

`GraphSnapshot::statistics()` returns exact node/edge row counts, original input batch counts and the logical bytes added for UInt64 relationship ordinals. These are cached during capture and read in constant time without a provider scan or graph export. Cloned snapshots keep identical statistics; session catalog replacement cannot change them. Batch counts do not claim execution parallelism. Selectivity, join cardinalities, serialized graph size and total memory are not inferred from these counts. Automatic routing still requires qualified costs and complete resource admission.

18.5.7 Borrowed graph serialization (Ostracod 0.18.0)

`ArrowGraphTables::as_serializable_graph()` presents the ordinary Grust graph serde contract directly over native Arrow 55/58/59 columns. It preserves row and batch order, identity, sorted property keys, scalar tags, and missing versus null properties. It builds only per-batch column descriptors, without row graphs, property maps or string-value copies. The caller selects the serializer and sink; a bounded writer over `std::io::sink()` can count exact JSON bytes without keeping the encoded output. Writer errors propagate and may leave a prefix in a real sink. This enables native input-size admission; automatic routing and full query resource-policy integration remain separate work.

18.5.8 Native Arrow input admission (Ostracod 0.18.0)

`GraphSnapshot::try_new_with_input_policy` checks a prepared request's node/edge limits and exact graph JSON size before capturing native providers. It uses the borrowed Arrow serialization view and bounded counting writer, with no row graph or encoded JSON buffer. Row rejection precedes serialization; the original request deadline also covers capture. `SnapshotStatistics::serialized_graph_bytes` is `Some(exact_bytes)` after measured capture and `None` for ordinary capture, including an unmeasured empty graph. Clones retain the measurement.

`PreparedReadRequest::check_serializable_graph` shares these checks with trusted native adapters. `check_measured_graph` reuses exact cached measurements for the same immutable projection; estimates are not admissible substitutes. Backend snapshot authority, pre-existing

input buffers, ordinal allocation and execution work/intermediates remain separate obligations. This is input-size admission, not a complete bounded DataFusion executor or automatic routing.

18.5.9 Owned buffer admission (Brine 0.20.0)

`retain_buffer_owner` attaches an application token to an immutable Arrow buffer through safe `bytes::Bytes` ownership. Native slices, array clones and C Data exports retain that owner without copying bytes. Existing clones made before attachment do not acquire the token. Reserve storage before allocation; this helper preserves ownership and does not itself measure memory.

`GraphSnapshot::try_new_with_context` admits the added UInt64 ordinal payload and construction work. Reservations stay with the buffers after the snapshot, provider or result stream disappears. The combined `try_new_with_input_policy_and_context` also checks exact input size and requires an execution deadline no later than the prepared request's deadline. Input buffers, provider/schema metadata and allocator capacity remain outside the logical ordinal-payload bound. These capture checks do not establish backend authority, full operator accounting or automatic execution selection.

`retain_array_owner` extends buffer ownership through an array's validity bits and nested children. It rebuilds and validates metadata without copying payload bytes. Arrays with no physical buffers retain no token; a wrapper remains necessary when metadata-only lifetime must be admitted. Algorithm result batches use the same mechanism so raw batch clones and retained child slices keep their original reservation. New reservations are not charged for each clone.

Conclusion

Grust is small by design. Its core model is easy to hold in your head, and its backend contract is narrow enough that very different systems can implement it. That is the source of its leverage.

The project says: build your graph once, keep the domain model in Rust, and let the backend translate. Sometimes that translation is a map scan. Sometimes it is Arrow and LanceDB. Sometimes it is PostgreSQL, Spark SQL, Redis graph commands, SurrealQL, Helix SDK calls, or CocoIndex target state. And sometimes it is a capability-secured agent memory path in which TypeSec guards authority while Grust persists opaque records and their entity graph durably in Turso.

The more Grust grows, the more important that center becomes: a stable property graph model, a backend-neutral traversal IR, explicit errors, feature-gated integrations, and enough Rust type structure to make the right path feel natural.